数据结构

dsu on tree

```
vector<int> G[N];
// siz 子树大小 lef,rig dfs序子树区间 pos[u]u在dfs序的位置， heavy重儿子 idx dfs序
int siz[N], lef[N], pos[N], rig[N], heavy[N], dfn;
int n;
// 预处理信息
void dfs1(int u, int fa)
{
    siz[u]=1;   //子树大小
    lef[u]=++dfn;pos[dfn]=u;// dfs序
    for(auto v : G[u]) if(v != fa){
        dfs1(v, u);
        siz[u] += siz[v];
        if(heavy[u] == -1 || siz[v] > siz[heavy[u]])    // 重儿子
            heavy[u] = v;
    }
    rig[u]=dfn;
}
void dfs2(int u, int fa, int keep)
{
    for(auto v : G[u])      // 先计算所有轻儿子 并且不保存信息
        if(v != fa && v!= heavy[u])
            dfs2(v, u, 0);

    if(heavy[u] != -1)// 不是叶子
        dfs2(heavy[u], u, 1);   // 计算重儿子 保存重儿子信息

    auto add = [&](int u){
        /*添加函数*/
    };
    auto del = [&](int u){
        /*删除函数*/
    };

    // 轻儿子加入重儿子  暴力将轻儿子信息加入
    for(auto v : G[u])
        if(v != fa && v!= heavy[u]){
            For(i,lef[v],rig[v]){   // 用dfs序减少递归的时间消耗
                add(pos[i]);
```

```
            }
        }
    //加入自己
    add(u);
    //  如果不保存信息 就清空删除
    if(!keep){
        // maxcnt = sumcnt = 0;
        // For(i,lef[u],rig[u])
        //  del(pos[i]);
    }
}
```

树链剖分

```
vector<int> G[N];
// hson重孩  top[u]点u所在重链的最上面点 dep深度 fa父亲 siz子树大小
int n, hson[N], top[N], dep[N], fa[N], siz[N];
// dfs2才做dfs序
int l[N], r[N], dfn, pos[N];
void dfs1(int u, int f=0)
{
    siz[u] = 1;
    hson[u] = -1;
    fa[u] = f;
    for(auto v : G[u]) if(v != f){
        dep[v] = dep[u] + 1;
        dfs1(v, u);
        siz[u] += siz[v];
        if(hson[u] == -1 || siz[hson[u]] < siz[v])
            hson[u] = v;
    }
}
void dfs2(int u, int t)  // t 重链头
{
    top[u] = t;
    l[u] = ++dfn;   // 同一重链的dfs序是连续的
    pos[dfn] = u;
    if(hson[u] != -1)   //如果有重孩子
        dfs2(hson[u], t);   // 同一条重链

    for(auto v:G[u]){
        if(v != fa[u] && v != hson[u])
            dfs2(v, v); // 通过轻边连接
    }
    r[u] = dfn;
}
```

```
int LCA(int u, int v)    // lca过程中可以整段重链信息一起查询
{
    // 跳到一条重链
    while(top[u] != top[v])
    {
        if(dep[top[u]] < dep[top[v]]){
            v = fa[top[v]];
        }
        else{
            u = fa[top[u]];
        }
    }
    if(dep[u] < dep[v])return u;
    return v;
}
```

虚树

```
int k = 0, m = 0, tot = 0;
cin >> k;
For(i,1,k){ // k个关键点
    cin >> b[i];
    imp[b[i]] = 1;  // b[i]是关键点
}
auto cmp = [&](int a, int b){return dfn[a]<dfn[b];};
m = tot = k;
sort(b+1, b+1+tot, cmp);          // dfs序排序
For(i,2,m) b[++tot] = LCA(b[i], b[i-1]);     // 相邻关键点的lca
b[++tot] = 1;    // 加根节点
sort(b+1, b+1+tot, cmp);
tot = unique(b+1, b+1+tot)-b-1; // 去重
For(i,1,tot-1){
    int lc = LCA(b[i], b[i+1]);
    H[lc].push_back(b[i+1]);     // 建树 上到下的边
}
dfs(1); // dfs根节点能跑到整个虚树
/***work***/
// 清空
For(i,1,tot){
    H[b[i]].clear();
    imp[b[i]] = 0;
}
```

树哈希

无权树

```
ll H(ll x) {
    return x * x * x * 1237123 + 19260817;
}
ll F(ll x) {
    ll cur = H(x & ((1 << 31) - 1)) + H(x >> 31);
    return cur;
}
```

h_u为u的子树的哈希值

$$h_u = 1 + \sum_{v \in son(u)} F(h_v)$$

有权树（不一定对）

$$f_u = size_u \times w_u \times \sum_{v \in son(u)} f_v \times seed^{i-1}$$

u的第i个孩子是v。

环形涂色问题

将一个圆分成$n(n \geq 2)$个扇形区域，用$m(m \geq 2)$中不同的颜色染色，要求相邻两个扇形区域颜色不同，染色方案数为$(-1)^n(m-1)+(m-1)^n$。
证明: $m(m-1)^{n-1}$包含了第一个和最后一个颜色一样的情况，当第一个和最后一个颜色一样时即为a_{n-1}

即: $a_n + a_{n-1} = m(m-1)^{n-1}$，同乘$(-1)^n$，
$(-1)^n a_n + (-1)^n a_{n-1} = (-1)^n m(m-1)^{n-1}$，将$n=3$到$n=n$的式子加起来得到左边为$(-1)^n a_n - a_2$，右边是一个等比数列求和然后再搞搞就o了

主席树(静态区间第k小)

```

#include<bits/stdc++.h>
#define For(i,a,b) for(int i = a; i <= b; i++)
using namespace std;
const int N = 2e7+10;
struct Node{
```

```
    int ls, rs, num;
}tr[N];
int rt[N], cnt;
void pushup(int u) {
    tr[u].num = tr[tr[u].ls].num+tr[tr[u].rs].num;
}
void build(int &u, int l, int r)
{
    u = ++cnt;
    if(l==r)return;
    int mid = (l+r)>>1;
    build(tr[u].ls, l, mid);
    build(tr[u].rs, mid+1, r);
    pushup(u);
}
void add(int &u, int pre, int l, int r, int pos, int x)
{
    u = ++cnt;       // 新建节点
    tr[u] = tr[pre];     // 继承原节点的信息
    if(l==r){
        assert(pos == l);
        tr[u].num+=x;
        return;
    }
    int mid = (l+r)>>1;
    // 递归下含有pos的那个子树
    if(pos <= mid)add(tr[u].ls, tr[pre].ls, l, mid, pos, x);
    else add(tr[u].rs, tr[pre].rs, mid+1, r, pos, x);
    pushup(u);
}
int n, a[N], q;
int query(int u, int v, int l, int r, int k)
{
    // cout << u << ' ' << v << ' ' << l << ' ' << r << ' ' << tr[u].num << ' ' << tr[v].num << '\n';
    if(l==r)return l;
    int mid = (l+r)>>1;
    int lsnum = tr[tr[u].ls].num-tr[tr[v].ls].num;
    if(lsnum >= k) return query(tr[u].ls, tr[v].ls, l, mid, k);
    else return query(tr[u].rs, tr[v].rs, mid+1, r, k-lsnum);
}
signed main()
{
    cin >> n >> q;
    vector<int> li;
    For(i,1,n){
        cin >> a[i];
        li.push_back(a[i]);
    }
```

```
    sort(li.begin(), li.end()); // 离散化
    li.erase(unique(li.begin(), li.end()), li.end());
    int m = li.size();
    build(rt[0], 1, m);      // 原数组的线段树
    For(i,1,n){
        int pos = lower_bound(li.begin(), li.end(), a[i])-li.begin()+1;
        add(rt[i], rt[i-1], 1, m, pos, 1);  // 插一个数一个版本
    }
    while(q--){
        int l, r, k;
        cin >> l >> r >> k;
        // 将r的树和l-1的树做差
        cout << li[query(rt[r], rt[l-1], 1, m, k)-1] << '\n';
    }
}
```

树状数组套主席树

P3380 【模板】二逼平衡树（树套树）：

您需要写一种数据结构（可参考题目标题），来维护一个有序数列，其中需要提供以下操作：

1. 查询 k 在区间内的排名
2. 查询区间内排名为 k 的值
3. 修改某一位值上的数值
4. 查询 k 在区间内的前驱（前驱定义为严格小于 x，且最大的数，若不存在输出 `-2147483647`）
5. 查询 k 在区间内的后继（后继定义为严格大于 x，且最小的数，若不存在输出 `2147483647`）

```
//  P3380 【模板】二逼平衡树（树套树）
#include<bits/stdc++.h>     // 加上离散化
#define ll long long
#define For(i,a,b) for(int i = a; i <= b; ++i)
using namespace std;
const int N = 5e5;
int lowbit(int x){return x&-x;}
struct Node{
    int ls, rs, val;
}tr[N<<5];
vector<int> li;
int n, q, a[N], rt[N], m, cnt, b[N];
int nodeR[N], nodeL[N], cntR, cntL;
void pushup(int u)
{
    tr[u].val = tr[tr[u].ls].val + tr[tr[u].rs].val;
}
void _modify(int &u, int l, int r, int pos, int val)
{
```

```
    if(!u)u = ++cnt;     // 如果本来没点就开
    if(l==r){
        tr[u].val += val;
        return;
    }
    int mid = (l+r)>>1;
    if(pos <= mid) _modify(tr[u].ls, l, mid, pos, val);
    else _modify(tr[u].rs, mid+1, r, pos, val);
    pushup(u);
}
// op=3     a[pos] += val
void add(int pos, int val, int num)
{
    while(pos <= n) {    // 树状数组那些节点的线段树
        _modify(rt[pos], 0, m, num, val);
        pos += lowbit(pos);
    }
}
// op=1     k在[l,r]的排名
int _query_kRank(int l, int r, int k)
{
    if(l==r)return 0;        // 同数字的不算数
    int mid = (l+r)>>1, lsum = 0;
    if(k<=mid){
        For(i,1,cntR) nodeR[i] = tr[nodeR[i]].ls;
        For(i,1,cntL) nodeL[i] = tr[nodeL[i]].ls;
        return _query_kRank(l, mid, k);
    }
    else{
        For(i,1,cntR)lsum += tr[tr[nodeR[i]].ls].val, nodeR[i] = tr[nodeR[i]].rs;
        For(i,1,cntL)lsum -= tr[tr[nodeL[i]].ls].val, nodeL[i] = tr[nodeL[i]].rs;
        return lsum + _query_kRank(mid+1, r, k);
    }
}
// op=2     第k名
int _query_kth(int l, int r, int k)
{
    if(l==r)return l;
    int mid = (l+r)>>1;
    int lsum = 0;
    For(i,1,cntR)lsum += tr[tr[nodeR[i]].ls].val;
    For(i,1,cntL)lsum -= tr[tr[nodeL[i]].ls].val;
    if(lsum >= k){
        For(i,1,cntL) nodeL[i] = tr[nodeL[i]].ls;
        For(i,1,cntR) nodeR[i] = tr[nodeR[i]].ls;
        return _query_kth(l, mid, k);
    }
```

```
    else{
        For(i,1,cntL) nodeL[i] = tr[nodeL[i]].rs;
        For(i,1,cntR) nodeR[i] = tr[nodeR[i]].rs;
        return _query_kth(mid+1, r, k-lsum);
    }
}
void get_Root(int l, int r)
{
    cntR = cntL = 0;
    for(int i = l-1; i; i -= lowbit(i))
        nodeL[++cntL] = rt[i];
    for(int i = r; i; i -= lowbit(i))
        nodeR[++cntR] = rt[i];
}
struct Que{
    int op, l, r, k;
}que[N];
signed main()
{
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin >> n >> q;
    For(i,1,n){
        cin >> a[i];
        li.push_back(a[i]);
    }
    For(i,1,q){
        cin >> que[i].op;
        if(que[i].op == 3)cin >> que[i].l >> que[i].k;
        else cin >> que[i].l >> que[i].r >> que[i].k;
        li.push_back(que[i].k);
    }
    sort(li.begin(), li.end());
    li.erase(unique(li.begin(), li.end()), li.end());
    m = li.size();
    For(i,1,n){
        b[i] = lower_bound(li.begin(), li.end(), a[i])-li.begin()+1;
        add(i, 1, b[i]);
    }
    For(i,1,q){
        auto [op, l, r, k] = que[i];
        if(op == 3){    // 修改 a[l] = k
            add(l, -1, b[l]);       // 将原本删掉
            b[l] = lower_bound(li.begin(), li.end(), k)-li.begin()+1;
            add(l, 1, b[l]);
        }
        else {
            get_Root(l, r);
            if(op == 1) {       // 查区间[l, r] k的排名
                k = lower_bound(li.begin(), li.end(), k)-li.begin()+1;
```

```
                cout << _query_kRank(0, m, k)+1 << '\n';
            }
            else if(op == 2)    // 查区间[l, r]第k名
                cout << li[_query_kth(0, m, k)-1] << '\n';
            else if(op == 4){   // 查区间[l, r] <k的最大数字
                k = lower_bound(li.begin(), li.end(), k)-li.begin()+1;
                int rk = _query_kRank(0, m, k);
                if(rk == 0)cout << "-2147483647\n";
                else {
                    get_Root(l, r);
                    cout << li[_query_kth(0, m, rk)-1] << '\n';
                }
            }
            else if(op == 5){       // 查区间[l, r] >k的最小数字
                k = lower_bound(li.begin(), li.end(), k)-li.begin()+1;
                int rk = _query_kRank(0, m, k+1);   // k+1能返回<=k的数量
                if(rk == r-l+1)cout << "2147483647\n";
                else {
                    get_Root(l, r);
                    cout << li[_query_kth(0, m, rk+1)-1] << '\n';
                }
            }
            else assert(0);
        }
    }
    return 0;
}
```

李超线段树

支持插入一条线段，每次查询k，返回线段和直线$x = k$的交点中最大的y

```
#include<bits/stdc++.h>
using namespace std;
typedef pair<double,int> pdi;
const int N = 1e5+10, mod = 39989, inf = 0x3f3f3f3f;
#define EPS 1e-9
struct Line{
    // 斜率为0时直接 k=0 b=max(y)
    double k, b;    // y = kx+b;
}line[N];
double f(int id, int x){return line[id].k*x + line[id].b;}
int sign(double a){return a < -EPS ? -1 : a > EPS;}
int cmp(double a, double b){return sign(a-b);}
struct LiChao{
    int tr[N<<2];   // tr[u]为u节点区间中电最优线段的下标
```

```
    #define lson (u<<1)
    #define rson (u<<1|1)
    // 插入下标为id的直线
    void insertLine(int u, int l, int r, int id){
        int &uid = tr[u], mid = (l+r)>>1;
        int midcmp = cmp(f(id, mid), f(uid, mid));
        // 如果新插入线段更优 || 相等&&编号更小，则u的最优线段为插入线段，原线段下传
        if(midcmp > 0 || (midcmp == 0 && id < uid))
            swap(id, uid);
        int lcmp = cmp(f(id, l), f(uid, l)), rcmp = cmp(f(id, r), f(uid, r));
        // 递归将id插入可能是最优线段的子节点中，最多只有1个if是成立的
        if(lcmp > 0 || (lcmp == 0 && id < uid)) insertLine(lson, l, mid, id);
        if(rcmp > 0 || (rcmp == 0 && id < uid)) insertLine(rson, mid+1, r, id);
    }
    // 插入下标为id 区间为[il, ir]的线段
    void insertSeg(int u, int l, int r, int il, int ir, int id){
        if(il <= l && r <= ir){ // 当前区间被线段完全覆盖，插
            insertLine(u, l, r, id);
            return;
        }
        int mid = (l+r)>>1;
        if(il <= mid) insertSeg(lson, l, mid, il, ir, id);
        if(ir > mid) insertSeg(rson, mid+1, r, il, ir, id);
    }
    // pair的比较函数
    pdi pmax(pdi x, pdi y){
        if(cmp(x.first, y.first) > 0)return x;
        else if(cmp(x.first, y.first) < 0)return y;
        else if(x.second < y.second)return x;
        else return y;
    }
    // 查询x=x的直线上最高的点在那个线段上，返回{y, id}
    pdi query(int u, int l, int r, int x){
        if(r < x || l > x) return {-inf, 0};    // 线段为空返回0
        if(l==r) return {f(tr[u], x), tr[u]};
        int mid = (l+r)>>1;
        double value = f(tr[u], x);
        return pmax({value, tr[u]},
            pmax(query(lson, l, mid, x), query(rson, mid+1, r, x)));
    }
}T;
signed main()
{
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    int q, lastans = 0, cnt = 0;
    cin >> q;
```

```
    while(q--){
        int op, k, x[2], y[2];
        cin >> op;
        if(op==0){
            cin >> k;
            k = (k+lastans-1)%mod+1;
            lastans = T.query(1,1,mod,k).second;
            cout << lastans << '\n';
        }
        else{
            const int M = 1000000000;
            for(int i = 0; i <= 1; i++){
                cin >> x[i] >> y[i];
                x[i] = (x[i]+lastans-1)%mod+1;
                y[i] = (y[i]+lastans-1)%M+1;
            }
            if(x[0] > x[1]){
                swap(x[0], x[1]);
                swap(y[0], y[1]);
            }
            ++cnt;
            if(x[1] == x[0]){    // 特判斜率不存在
                line[cnt] = {0, (double)max(y[1], y[0])};
            }
            else{
                double kk = 1.0*(y[1]-y[0])/(x[1]-x[0]);
                line[cnt] = {kk, y[0]-kk*x[0]};
            }
            T.insertSeg(1,1,mod,x[0],x[1],cnt);
        }
    }
    return 0;
}
```

楼房重建线段树(P4198)

题意：在一个平面内，求出所有的高度大于0的点与$(0,0)$的连线没有与之前的线段相交的楼房，单点修改。

思路：将高度转化为斜率，变成求从1开始的最长上升序列。线段树维护区间最大值和区间答案。

单点修改考虑如何合并2个子树，左子树答案+当前为k在右子树的上升序列，复杂度为$O(\log^2 n)$。

```
int query(int u, int l, int r, double k)
{
    if(k==0)return tr[u].val;
    if(l==r)return tr[u].mx > k;
```

```
    if(tr[u].mx <= k)return 0;
    int mid = (l+r)>>1;
    // 如果左孩最大值小于k 只查右
    if(tr[lson].mx <= k) return query(rson, mid+1, r, k);
    else {
        // 否则查左 + 右孩贡献
        return query(lson, l, mid, k)+tr[u].val-tr[lson].val;
    }
}
void modify(int u, int l, int r, int pos, double val)
{
    if(l>pos || r<pos)return;
    if(l==r){
        tr[u] = {val, 1};
        return;
    }
    int mid = (l+r)>>1;
    modify(lson, l, mid, pos, val);
    modify(rson, mid+1, r, pos, val);
    tr[u].mx = max(tr[lson].mx, tr[rson].mx);    // 区间最大值
    // 左子树答案 + 最大斜率tr[lson].mx 查右子树
    tr[u].val = tr[lson].val + query(rson, mid+1, r, tr[lson].mx);
}
```

线段树合并/分裂

- 线段树合并，将v的树合并到u上。复杂度为u, v两棵树一样的节点个数

```
void merge(int &u, int v, int l, int r)
{
    if(!v)return;    // v树为空
    if(!u){ // u树为空，当前节点直接是v
        u = v;
        return;
    }
    if(l==r){    // 合并节点信息
        tr[u].cnt += tr[v].cnt;
        tr[u].val = l;
        return;
    }
    // 暴力合并左右子树
    int mid = (l+r)>>1;
    merge(ls(u), ls(v), l, mid);
    merge(rs(u), rs(v), mid+1, r);
    update(u);
}
```

- 线段树分裂，将v树的区间$[l, r]$，分裂出新的树。

```
void split(int& u, int& v, int l, int r, int ql, int qr)
{
    if(ql <= l && r <= qr){ // 当前区间为要分裂的区间
        u = v;
        v = 0;  // v树删掉这个节点
        return;
    }
    if(!v)return;
    if(!u)u = ++tot;
    int mid = (l+r)>>1;
    if(ql <= mid)split(ls(u), ls(v), l, mid, ql, qr);
    if(qr > mid)split(rs(u), rs(v), mid+1, r, ql, qr);
    update(u);
    update(v);
}
```

二维树状数组

```
// op为更新树状数组的方法 可以是add, max, min...
template<class T, T(*op)(T, T)>
class BIT2D{
    T tr[N][N];
    int lowbit(int x){return x&-x;}
public:
    BIT2D(){memset(tr, 0, sizeof(tr));}
    BIT2D(T x){
        For(i,0,N-1)For(j,0,N-1)tr[i][j] = x;
    }
    void add(int x, int y, T d){    // (x, y)这点+=d
        while(x<N){
            for(int i = y; i < N; i += lowbit(i))
                tr[x][i] = op(tr[x][i], d);
            x += lowbit(x);
        }
    }
    T query(int x, int y, T ori){
        T res = ori;
        while(x){
            for(int i = y; i; i -= lowbit(i))
                res = op(res, tr[x][i]);
            x -= lowbit(x);
        }
        return res;
    }
```

```
};
```

K-D Tree

```
struct P{
    int x, y, val;
}ps[N];
struct Node
{
    int sum, sz;     // 总和，点数
    int ls, rs, d;   // 左右孩，根据第d维划分(这里是x/y)
    int L, R, D, U; // 这个节点管的边界
}tr[N];
int cnt, arr[N];     // 重构时用
#define lson (tr[u].ls)
#define rson (tr[u].rs)
int n, idx, root;
double alpha = 0.8; // 有一个子树大小占>=alpha就重构
bool isbad(int u){
    return max(tr[lson].sz, tr[rson].sz) >= alpha*tr[u].sz;
}
void update(int u)  // 更新u结点的信息
{
    tr[u].sz = tr[lson].sz + tr[rson].sz + 1;    // 还有u自己
    // 开新点的时候u的编号就是这个点的下标
    tr[u].sum = tr[lson].sum + tr[rson].sum + ps[u].val;
    tr[u].L = tr[u].R = ps[u].x;
    tr[u].D = tr[u].U = ps[u].y;
    if(lson){    // 有左孩
        tr[u].L=min(tr[u].L,tr[lson].L),tr[u].R=max(tr[u].R,tr[lson].R);
        tr[u].D=min(tr[u].D,tr[lson].D),tr[u].U=max(tr[u].U,tr[lson].U);
    }
    if(rson){    // 有右孩
        tr[u].L=min(tr[u].L,tr[rson].L),tr[u].R=max(tr[u].R,tr[rson].R);
        tr[u].D=min(tr[u].D,tr[rson].D),tr[u].U=max(tr[u].U,tr[rson].U);
    }
}
bool cmpx(int a, int b){return ps[a].x < ps[b].x;}
bool cmpy(int a, int b){return ps[a].y < ps[b].y;}
void middfs(int u){
    if(!u)return;
    middfs(lson);
    arr[++cnt] = u;
    middfs(rson);
}
int build(int l, int r){
```

```
    if(l>r)return 0;
    int mid = (l+r)>>1;
    // 计算横纵坐标的方差，谁大按谁分
    double avg1 = 0, avg2 = 0, var1 = 0, var2 = 0;
    For(i,l,r){
        avg1 += ps[arr[i]].x;
        avg2 += ps[arr[i]].y;
    }
    avg1 /= (r-l+1), avg2 /= (r-l+1);
    For(i,l,r){
        var1 += (avg1 - ps[arr[i]].x)*(avg1 - ps[arr[i]].x);
        var2 += (avg2 - ps[arr[i]].y)*(avg2 - ps[arr[i]].y);
    }
    if(var1 > var2){
        nth_element(arr+l, arr+mid, arr+r+1, cmpx);
        tr[arr[mid]].d = 1;
    }
    else{
        nth_element(arr+l, arr+mid, arr+r+1, cmpy);
        tr[arr[mid]].d = 2;
    }
    int u = arr[mid];
    lson = build(l, mid-1);
    rson = build(mid+1, r);
    update(u);
    return u;
}
void rebuild(int& u){
    cnt = 0;
    middfs(u);
    u = build(1, cnt);
}
void insert(int& u, int id)
{
    if(!u){ // 当前节点位空
        u = id;
        update(u);
        return;
    }
    if(tr[u].d == 1){    // 以x为分界线的
        if(ps[id].x <= ps[u].x) insert(tr[u].ls, id);    // 在分界线右边
        else insert(tr[u].rs, id);
    }
    else{
        if(ps[id].y <= ps[u].y) insert(tr[u].ls, id);
        else insert(tr[u].rs, id);
    }
    update(u);
    if(isbad(u))rebuild(u); // 如果当前结点过于不平衡，重构该子树
```

```
}
#define y1 alskdjalskdjaslkdj
int x1, x2, y1, y2;
int query(int u)     // 查询矩形内点的权值和
{
    if(!u || x2<tr[u].L || x1>tr[u].R || y1>tr[u].U || y2<tr[u].D)return 0;
    if(x1<=tr[u].L && tr[u].R<=x2 && y1<=tr[u].D && tr[u].U<=y2)return tr[u].sum;
    int res = 0;
    if(x1 <= ps[u].x && ps[u].x <= x2 && y1 <= ps[u].y && ps[u].y <= y2)
        res = ps[u].val;
    return query(lson) + query(rson) + res;
}
signed main()
{
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin >> n;
    // 可以用类似rebuild的方法先build n个点
    int lastans = 0, op, x, y, val;
    while(cin >> op){
        if(op==1){
            cin >> x >> y >> val;
            x ^= lastans, y ^= lastans, val ^= lastans;
            ps[++idx] = {x, y, val};
            insert(root, idx);
        }
        else if(op==2){
            cin >> x1 >> y1 >> x2 >> y2;
            x1 ^= lastans, x2 ^= lastans, y1 ^= lastans, y2 ^= lastans;
            lastans = query(root);
            cout << lastans << '\n';
        }
        if(op==3)break;
    }
    return 0;
}
```

三维偏序(CDQ分治)

n个点，每个点有一个坐标(a_i, b_i, c_i) (可能会有重点), 对于每个点i求有多少个j满足$a_j \le a_i$, $b_j \le b_i, c_j \le c_i, j \ne i$。

```
#include<bits/stdc++.h>
#define ll long long
#define int ll
```

```cpp
#define For(i,a,b) for(int i = a; i <= b; ++i)
using namespace std;
const int N = 2e5+10;
BIT<int> T;
struct Node{
    int x, y, z, cnt, ans; // cnt:重合点数, ans:这个点的答案
    bool operator < (const Node& b)const{
        return array<int, 3>{x, y, z} < array<int, 3>{b.x, b.y, b.z};
    }
    bool operator == (const Node& b){
        return !((*this)<b)&&!(b<(*this));
    }
}a[N], p[N];
bool cmpy(const Node& a, const Node& b){
    return array<int, 2>{a.y, a.z} < array<int, 2>{b.y, b.z};
}
int n, k, ans[N];
void cdq(int l, int r)
{
    if(l==r)return;
    int mid = (l+r)>>1, pos = l;
    cdq(l, mid);cdq(mid+1, r);
    // 合并两边(第二维)
    sort(p+l, p+mid+1, cmpy);
    sort(p+mid+1, p+r+1, cmpy);
    // 树状数组做第三维
    For(i, mid+1, r){
        // 把可以的第二维都进树状数组
        while(pos <= mid && p[pos].y <= p[i].y){
            T.add(p[pos].z, p[pos].cnt);
            pos++;
        }
        p[i].ans += T.query(p[i].z);
    }
    For(i,l,pos-1)T.add(p[i].z, -p[i].cnt);
}
signed main()
{
    cin >> n >> k;
    int idx = 0;
    For(i,1,n) cin >> a[i].x>>a[i].y>>a[i].z;
    sort(a+1,a+1+n);    // 第一维排序
    p[++idx] = a[1];
    p[idx].cnt = 1;
    For(i,2,n){
        if(p[idx] == a[i])p[idx].cnt++;
        else{
            p[++idx] = a[i];
            p[idx].cnt = 1;
```

```
        }
    }
    cdq(1, idx);
    For(i,1,idx){
        ans[p[i].ans+p[i].cnt-1] += p[i].cnt;
    }
    For(i,0,n-1)cout << ans[i] << '\n';
}
```

点分治

```
int n, k;
vector<pii> G[N];
// rt是重心
int vis[N];
int rt, sz[N], wt[N];   // wt[u]是u作为rt时的重儿子的sz
void findRt(int u, int fa, int subSZ){  // subSZ是当前子树大小
    sz[u] = 1, wt[u] = 0;
    for(auto [v, w]:G[u])if(v != fa && !vis[v]){
        findRt(v, u, subSZ);
        sz[u] += sz[v];
        wt[u] = max(wt[u], sz[v]);
    }
    wt[u] = max(wt[u], subSZ - sz[u]);
    if(wt[rt] > wt[u])rt = u;   // 找个重儿子最小的作为根
}
int arr[N], cnt;
void dfs(int u, int fa, int dep){
    arr[++cnt] = dep;
    for(auto [v, w]:G[u])if(!vis[v] && v != fa)
        dfs(v, u, dep+w);
}
// 算答案部分，点分治之后，算u子树内经过u的路径的答案
int calc(int u, int val){   // 计算子树内 路径+val <= k的对数, 路径可能是非法的
    cnt = 0;
    dfs(u, 0, val);
    sort(arr+1, arr+1+cnt);
    int res = 0;
    for(int l = 1, r = cnt; l <= r; l++){
        while(l<=r && arr[l]+arr[r]>k)r--;
        if(l>r)break;
        // l -> u -> (l,r]  有r-l条
        res += r-l;
    }
    return res;
}
```

```
int ans;
void DFS(int u){
    ans += calc(u, 0);
    vis[u] = 1;      // vis设为1，做子树的时候就不会dfs出来了
    for(auto [v, w]:G[u])if(!vis[v]){
        ans -= calc(v, w);  // 容斥 减掉前面calc(u, 0)多算的部分
        rt = 0;
        findRt(v, 0, sz[v]);   // 重新在v子树找rt
        DFS(rt);
    }
}
void solve()
{
    cin >> n;
    ans = rt = 0;
    wt[rt] = LINF;
    For(i,1,n-1){
        int u, v, w;
        cin >> u >> v >> w;
        G[u].push_back({v, w});
        G[v].push_back({u, w});
    }
    cin >> k;
    findRt(1, 0, n);
    DFS(rt);
    cout << ans << "\n";
}
```

点分树

通过点分治的方法搞一颗重构树，这棵树的最大深度为log，可以搞点暴力跳的东西

- **注意**：重构后的树边和对应多条原树树边
- 常用于解决与树原形态无关的带修改问题。

```
/*
树分治版题
给一棵树，每个点有点权， 多次操作(强制在线)
操作1：修改一个点的点权
操作2：查询离x 小于等于k的所有点权和
*/
#include<bits/stdc++.h>
#define ll long long
#define ull unsigned long long
#define int ll
#define fr first
#define se second
#define INF 0x3f3f3f3f
```

```cpp
#define LINF 0x3f3f3f3f3f3f3f3f
#define all(x) x.begin(),x.end()
#define For(i,a,b) for(int i = a; i <= b; ++i)
#define Rep(i,a,b) for(int i = a; i >= b; --i)
using namespace std;
typedef pair<int,int> pii;
const int N = 2e5+10;
template<class T>
struct BIT{
    vector<T> tre;
    BIT(){}
    int lowbit(int x){return x&-x;}
    void add(int x, T d) {
        while(x < (ll)tre.size()) {
            tre[x] += d;
            x += lowbit(x);
        }
    }
    T query(int x) {
        x = min(x, (ll)tre.size()-1);
        T res = 0;
        while(x) {
            res += tre[x];
            x -= lowbit(x);
        }
        return res;
    }
};
// G1原树 G重构树
vector<int> G1[N], G[N];
BIT<ll> T[2][N];    // BIT用vector动态开时 查询修改记得取min
int fa[N][21], dep[N];
int cfa[N];      // 重构树上的fa
int n, a[N], q;
int rt, sz[N], vis[N], wt[N];   // wt[u]是u作为rt时的重儿子的sz
namespace LCASP{
void dfs0(int u, int f){
    fa[u][0] = f;
    For(i,1,20)fa[u][i] = fa[fa[u][i-1]][i-1];
    for(auto v:G1[u])if(v != f){
        dep[v] = dep[u]+1;
        dfs0(v, u);
    }
}
int lca(int u, int v){
    if(dep[u] < dep[v])swap(u, v);
    int cha = dep[u] - dep[v];
    Rep(i,20,0)if(cha>>i&1)
        u = fa[u][i];
```

```
    if(u==v)return u;
    Rep(i,20,0) if(fa[u][i] != fa[v][i]){
        u = fa[u][i], v = fa[v][i];
    }
    return fa[u][0];
}
int dis(int u, int v){
    int lc = lca(u, v);
    return dep[u]+dep[v]-2*dep[lc];
}
}
using namespace LCASP;
void findRt(int u, int fa, int subSZ){  // subSZ是当前子树大小
    sz[u] = 1, wt[u] = 0;
    for(auto v:G1[u])if(v != fa && !vis[v]){
        findRt(v, u, subSZ);
        sz[u] += sz[v];
        wt[u] = max(wt[u], sz[v]);
    }
    wt[u] = max(wt[u], subSZ - sz[u]);
    if(wt[rt] > wt[u])rt = u;   // 找个重儿子最小的作为根
}
int tsz[N];
void getsz(int u, int fa){
    tsz[u] = 1;
    for(auto v:G1[u])if(v != fa && !vis[v]){
        getsz(v, u);
        tsz[u] += tsz[v];
    }
}
void dfs(int u) {
    getsz(u, 0);    // 用findRt的sz是不准的(只是在点分治时负责度对)
    T[0][u].tre.resize(tsz[u]+2, 0);
    T[1][u].tre.resize(tsz[u]+2, 0);
    vis[u] = 1;
    for(auto v:G1[u])if(!vis[v]) {
        rt = 0;
        findRt(v, u, tsz[v]);
        cfa[rt] = u;
        G[u].push_back(rt); // 建重构树，这题没用到
        dfs(rt);
    }
}
void solve()
{
    cin >> n >> q;
    For(i,1,n)cin >> a[i];
    For(i,2,n) {
        int u, v;
```

```
        cin >> u >> v;
        G1[u].push_back(v);
        G1[v].push_back(u);
    }
    dfs0(1, 0);

    rt = 0;
    wt[0] = LINF;
    findRt(1, 0, n);
    dfs(rt);

    auto add = [&](int u, int val){
        for(int c = u; c; c=cfa[c]){
            T[0][c].add(dis(u, c)+1, val);
        }
        for(int c = u; cfa[c]; c=cfa[c])T[1][c].add(dis(u, cfa[c])+1, val);
    };
    For(i,1,n) add(i, a[i]);

    int ans = 0;
    while(q--){
        int op, x, y;
        cin >> op >> x >> y;
        x ^= ans, y ^= ans;
        if(op == 0){
            ans = T[0][x].query(y+1);
            for(int u = x; cfa[u]; u = cfa[u]){
                int xdis = dis(cfa[u], x);
                if(y-xdis < 0)continue; // 这不能break 可能上面的点离x更近
                ans += T[0][cfa[u]].query(y-xdis+1);
                ans -= T[1][u].query(y-xdis+1);
            }
            cout << ans << "\n";
        } else {
            add(x, y-a[x]);
            a[x] = y;
        }
    }
}
signed main()
{
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    solve();
    return 0;
}
```

支配树

给定一个有向图，给定起点S，终点T。

- 支配点：在所有从S到T的路径中，必须经过的点(即各条路径上点集的交集）。

简而言之，如果删去某个点（即支配点），则不存在一条路径能够使S到达T。由支配点构成的树叫做支配树

支配树是一颗由起点S为根的树，根到树上任意一点路径经过的点都是它的支配点。

对于每颗子树，都符合上述性质（即子树内一点到该子树的根路径经过的点都是该子树的根的支配点）

使用`add_edge(u, v)`加有向边$u \to v$

`solve(root)`，以$root$为根求支配树

$idom[i] \to i$ 为支配树上的边

```
class DominatorTree{
public:
    std::vector <std::vector <int>> e, _e, tmp; // e原图 _e反图
    std::vector <int> dfn, inv; // dfn[x] x的dfs序
    int dfncnt;
    std::vector <int> sdom, idom;   // sdom[x] x的半支配点， idom[x] x的最近支配点
    std::vector <int> fa, father, value;

    explicit DominatorTree(int n):dfncnt(0){
        int sz = n + 10;
        e.resize(sz);
        _e.resize(sz);
        tmp.resize(sz);
        dfn.resize(sz);
        inv.resize(sz);
        sdom.resize(sz);
        idom.resize(sz);
        fa.resize(sz);
        father.resize(sz);
        value.resize(sz);
    }

    void add_edge(int u, int v){
        e[u].emplace_back(v);
        _e[v].emplace_back(u);
    }
```

```cpp

    int min(int u, int v){
        return dfn[u] < dfn[v] ? u : v;
    }

    int find(int u){
        if (fa[u] == u) return u;
        int f = fa[u];
        fa[u] = find(fa[u]);
        if (dfn[sdom[value[f]]] < dfn[sdom[value[u]]]) value[u] = value[f];
        return fa[u];
    }

    void dfs(int u){
        dfn[u] = ++ dfncnt;
        inv[dfncnt] = u;
        for (auto v : e[u]){
            if (dfn[v]){
                continue;
            }
            father[v] = u;
            dfs(v);
        }
    }

    void solve(int rt){
        dfs(rt);
        for (int i = 1; i <= dfncnt; ++ i){
            fa[inv[i]] = value[inv[i]] = sdom[inv[i]] = inv[i];
        }
        for (int i = dfncnt; i >= 2; -- i){
            int u = inv[i];
            for (auto v : _e[u]){
                if (!dfn[v]) continue;
                if (dfn[v] < i){
                    sdom[u] = min(sdom[u], v);
                }
                else{
                    find(v);
                    sdom[u] = min(sdom[u], sdom[value[v]]);
                }
            }
            fa[u] = father[u];
            tmp[sdom[u]].emplace_back(u);
            int pa = fa[u];
            for (auto v : tmp[pa]){
                find(v);
                idom[v] = value[v];
            }
```

```
            tmp[pa].clear();
        }
        for (int i = 2; i <= dfncnt; ++ i){
            int u = inv[i];
            idom[u] = sdom[u] == sdom[idom[u]] ? sdom[u] : idom[idom[u]];
        }
    }
};
```

数论、数学

公式

- 差比数列求和公式 $a_n=(an+b)q^{n-1}$, 则 $S_n=(An+B)q^n-B$, 其中 $A=\frac{a}{q-1}, B=\frac{b-A}{q-1}$
- 直角坐标系积分→极坐标(dr前有个r): $\iint\limits_D f(x,y)\,dx\,dy=\int\limits_\theta d\theta\int\limits_r f(r\cos\theta,r\sin\theta)rdr$

整数除法上下取整

```
ll flor(ll x,ll y)// x/y下取整
{
    if(y==0)exit(123);
    if(x%y==0)return x/y;
    if(x>0)return x/y;
    return x/y-1;
}
ll cil(ll x,ll y)// x/y上取整
{
    if(y==0)exit(123);
    if(x%y==0)return x/y;
    if(x<0)return x/y;
    return x/y+1;
}
```

gcd / exgcd

$ax+by=c$,(a,b为非0整数)

```
ll gcd(ll a, ll b) {
    if (b == 0) return a;
    else return gcd(b, a % b);
}
ll exgcd(ll a, ll b, ll &x, ll &y) {//返回gcd(a,b)
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
```

```
    return d;
}

// 解方程 ax+by=c   最小非负 x 解
int solve_equation(ll a, ll b, ll c, ll& x, ll& y)
{
    // ax+by=gcd(a,b)
    int g = exgcd(a, b, x, y);
    if(c % g != 0){     // 无解
        x = y = 0;
        return 0;
    }
    int t = b / g, k = a / g, d = c / g;
    x *= d;     // 将方程转为 ax+by=c
    x = (x % t + t) % t;
    y = (c - a * x) / b;
    /*
    // 最小非负 y
    y *= d;
    y = (y % k + k) % k;
    x = (c - b * y) / a;
    */
    return g;
}
```

ax ≡ b(mod m)的解

```
ll modequ(ll a, ll b, ll m)     //等价于ax+my=b
{
    ll x, y;
    ll d = exgcd(a, m, x, y);   // ax+my=gcd(a,m);
    if (b % d != 0) return -1;
    m /= d; a /= d; b /= d;     //a'x+m'y=d'   x=x*d' y=y*d'
    x = x * b % m;
    if (x < 0) x += m;
    return x;
}
```

类欧几里得 sum_{i=0~n} ⌊(ai+b)/(c)⌋

$$\sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor$$

```
int floor_sum(int a, int b, int c, int n) // O(log)
{
    int res = 0;
```

```
    if(a >= c)
    {
        res += n * (n + 1) * (a / c) / 2;
        a %= c;
    }
    if(b >= c)
    {
        res += (n + 1) * (b / c);
        b %= c;
    }
    int m = (a * n + b) / c;
    if(m == 0) return res;
    res += n * m - floor_sum(c, c - b - 1, a, m - 1);
    return res;
}
```

指数方程a^x ≡ b (mod m)的解(BSGS)

复杂度为$O(\sqrt{m})$ 用map多一个log

```
// 求解指数方程  a^x ≡ b (mod m) 最小x
ll Coprime_BSGS(ll a, ll b, ll m)// 要求: gcd(a, m)==1
{
    // a^(T*q) = b*a^r (mod m)
    assert(__gcd(a, m)==1);
    int T = sqrt(m)+2;
    ll v = ksm(a, T, m);
    unordered_map<ll,ll> mp;
    ll cur = 1;
    For(q,1,T){
        cur = cur*v%m;
        if(!mp.count(cur)) mp[cur] = q;
    }
    ll ans = m+1;
    cur = b;
    For(r,1,T){
        cur = cur*a%m;
        if(mp.count(cur)){
            ans = min(mp[cur]*T-r, ans);
        }
    }
    if(ans >= m)return -1;
    return ans;
}
ll BSGS(ll a, ll b, ll m) // (a, m)可以不互质
{
```

```
    // a^x = b (mod m)
    // a^M * a^(x-M) = b (mod m)
    // 要让 m/gcd(a^M, m) 与 a 互质, 大概是log
    // gcd(a^M,m) = gcd(a^M%m, m)    模掉自己gcd不变
    if(b==1)return 0;
    ll cur = 1, M = 30;
    For(i,1,M){
        cur = cur*a%m;
        if(cur==b) return i;
    }
    int g = __gcd(cur, m);
    if(b % g != 0)return -1;// 无解
    cur /= g, m /= g, b /= g;
    a %= m;
    b = modequ(cur, b, m);
    ll x = Coprime_BSGS(a, b, m);
    if(x == -1)return -1;
    return x + M;
}
```

合并两个同余方程

合并$x \equiv a(\mod b)$和$x \equiv c(\mod d)$

$$
\begin{aligned}
x &\equiv a(\mod b) \\
x &= bt + a \\
bt &\equiv c - a(\mod d) \\
bt &= -dy + c - a \\
bt + dy &= c - a
\end{aligned}
$$

```
// 初值  b=1, a=0
void merge(ll &a, ll &b, ll c, ll d) { // d <= 10^9
    if (a == -1 && b == -1)
        return;
    ll x, y;
    ll g = exgcd(b, d, x, y);   //bx+dy=gcd(b,d)
    //bx = g(mod d)
    if ((c - a) % g != 0) { // 无解
        a = b = -1;
        return;
    }
    d /= g; // d'   //理论上b/=g 但是后面用不到b
    ll t0 = ((c - a) / g) % d * x % d;
    if (t0 < 0) t0 += d;
    // t = t0 (mod d')
    a = b * t0 + a;
    b = b * d;
}
```

求幂塔a^{b...} mod p

当y充分大时（$y \geq \varphi(p)$）有:

$$x^y \mod p \to x^{y \mod \varphi(p)+\varphi(p)} \mod p$$

根据这条公式可以一直递归上去，递归$O(\log p)$轮就会变成 $\mod 1$。

当$y < \varphi(p)$直接做$x^y \mod p$就行了

```
int getphi(int x)
{
    if(mem.count(x))return mem[x];
    int res = x;
    for(int i = 2; i*i <= x; i++)if(x%i==0){
        res = res/i*(i-1);
        while(x%i==0)x/=i;
    }
    if(x!=1)res = res/x*(x-1);
    return res;
}
int solve(int l,int r,int m)
{
    int phi = getphi(m);    // 如果m是固定的最好预处理一下
    assert(phi!=0);
    if(l==r){
        return a[l];
    }
    if(phi==1)return 1;
    int res = solve(l+1, r, phi);
    int ans = ksm(a[l], res, phi);
    int flag=0;
    if(a[l]!=1){
        // 判断a[l]^res >= phi
        if(res>=30)flag=1;
        else if(pow(a[l],res)>1e9)flag=1;
        else{
            int asd = ksm(a[l],res,1000000007); // 没有意义的模数
            if(asd>=phi)flag=1;
        }
    }
    // 如果进入环了就要加上phi(m)
    if(flag)ans+=phi;
    return ans;
}
```

线性筛 / 线性筛筛积性函数

$pe[i]$表示i中最小值因子的指数次幂，即$pe_i = p_1^{a_1}$，p_1是唯一分解的最小素数

只要线性筛的话忽略第二个 For 和 pe 数组

```
// 线性筛 & 筛任意积性函数
// pe[i]表示i中最小值因子的指数次幂，即pe[i]=p1^a1
int p[N], prim[N], pe[N], tot, mu[N];
void init(){
    pe[1]=p[1]=1;
    For(i,2,N-1){
        if(!p[i])pe[i]=p[i]=prim[++tot]=i;
        for(int j = 1; j <= tot && prim[j]*i < N; j++){
            p[prim[j]*i] = prim[j];
            if(p[i]==prim[j]){
                pe[prim[j]*i] = pe[i]*prim[j];
                break;
            }
            pe[prim[j]*i] = prim[j];
            /*
            欧拉函数:
            if(prim[j]==p[i]){phi[i*prim[j]]=phi[i]*prim[j];break;}
            phi[i*prim[j]]=phi[i]*phi[prim[j]];
            莫比乌斯函数:
            if(prim[j]==p[i]){mu[i*prim[j]]=0;break;}
            mu[i*prim[j]]=-mu[i];
            */
        }
    }
    // 算积性函数
    mu[1]=1;
    For(i,2,N-1)
        if(pe[i] == i) {
            if(p[i]==i) mu[i] = -1; // 质数
            else mu[i] = 0; // 素数幂
        }
        else mu[i] = mu[pe[i]] * mu[i/pe[i]];   // 其他
}
```

积性函数

1. $id(x) = x$
2. $d(x)$ x的因子个数 $d(p^k) = k+1$
3. $\sigma(x)$ x的所有因子和 $\sigma(p^k) = p^0 + p^1 + \cdots + p^k = \frac{(1-p^{k+1})}{1-p} = \sigma(p^{k-1}) + p^k$ (等比数列求和)
4. $\varphi(n)$ 欧拉函数，计算与n互质的正整数之数目 $\varphi(p^k) = p^k(1-\frac{1}{p}) = p^k - p^{k-1}$

设$s(n)=\sum_{i=1}^{n}\varphi(i)$，有$\sum_{i=1}^{n}\sum_{j=i+1}^{n}[\gcd(i,j)=1]=s(n)$

$\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=1]=2s(n)-1$

还有$\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=k]=\sum_{i=1}^{\lfloor n/k\rfloor}\sum_{j=1}^{\lfloor n/k\rfloor}[\gcd(i,j)=1]$

5. $\mu(n)$ 莫比乌斯函数，$\mu(p^k)=1(k=0),-1(k=1),0(k\geq 2)$
6. $I(x)\leftrightarrow 1(x)=1$
7. $e(x)=(1(x=1),0(x\neq 1))=[x=1]$

$(*)$ 定义为狄利克雷卷积 $f(x),g(x)$ 为积性函数
$f*e=f$
$id*\mu=\varphi$
$1*\mu=e$
$f*1=g \iff f=g*\mu$
$g(n)=\sum_{d|n}f(d) \iff f(n)=\sum_{d|n}\mu(d)g(\frac{n}{d})$

亚线性筛法(杜教/min25)

杜教筛

用于求积性函数f的前n项和，时间复杂度$O(n^{\frac{2}{3}})$

- 记$S(n)=\sum_{i=1}^{n}f(i)$
- 需要构造一个积性函数g，满足:
  - 可以快速计算$\sum(f*g)(i)$
  - 可以快速计算$\sum g(i)$
- 记$h=f*g$

$$
\begin{aligned}
g(1)S(n)&=\sum_{i=1}^{n}g(i)S(\lfloor\frac{n}{i}\rfloor)-\sum_{i=2}^{n}g(i)S(\lfloor\frac{n}{i}\rfloor)\\
&=\sum_{i=1}^{n}(f*g)(i)-\sum_{i=2}^{n}g(i)S(\lfloor\frac{n}{i}\rfloor)
\end{aligned}
$$

代码有快速求φ 和 μ

```
int p[N], prim[N], tot, muPre[N], phiPre[N];
void init(){
    muPre[1] = phiPre[1] = 1;
    For(i,2,N-1){
        if(!p[i]){
            p[i] = prim[++tot] = i;
            muPre[i] = -1, phiPre[i] = i-1;
        }
```

```
        for(int j = 1; j <= tot && prim[j]*i < N; j++){
            p[prim[j]*i] = prim[j];
            if(p[i]==prim[j]){
                phiPre[i*prim[j]]=phiPre[i]*prim[j];
                muPre[i*prim[j]]=0;
                break;
            }
            phiPre[i*prim[j]]=phiPre[i]*phiPre[prim[j]];
            muPre[i*prim[j]]=-muPre[i];
        }
    }
    For(i,1,N-1){
        muPre[i] += muPre[i-1];
        phiPre[i] += phiPre[i-1];
    }
}
int sPhi(int n){
    // h(i) = i; g(i) = 1
    static map<int,int> mem;
    if(n < N)return phiPre[n];
    if(mem.count(n))return mem[n];
    int res = n*(n+1)/2;
    for(int l = 2, r; l <= n; l = r+1){
        int d = n/l;
        r = n/d;
        res -= (r-l+1)*sPhi(d);
    }
    return mem[n] = res;
}
int sMu(int n){
    // h(i) = [i==1]; g(i) = 1;
    static map<int,int> mem;
    if(n < N)return muPre[n];
    if(mem.count(n))return mem[n];
    int res = 1;
    for(int l = 2, r; l <= n; l = r+1){
        int d = n/l;
        r = n/d;
        res -= (r-l+1)*sMu(d);
    }
    return mem[n] = res;
}
```

min25筛

用于在低于线性复杂度求积性函数$\sum_{i=1}^{n} f(i)$

其中f要是积性函数，并且满足f一个多项式，$f(n) = \sum_{i=0}^{k} a_i n^i$，其中$a_i$是$n^i$的系数

分为2部分（时间复杂度时间复杂度是据说的，反正在10^{10-12}能过）

1. 第一部分`calcG`：算质数部分的f值，时间复杂度$O(\frac{n^{\frac{3}{4}}}{\log n})$，一般$n \le 10^{11}$可过，做质数和、质数个数、质数平方和，用第一部分就够了
2. 第二部分`S(n, 0)`：算整个$\sum f(n)$的答案，时间复杂度$O(n^{1-\epsilon})$

怎么灵活运用？看注释看命吧

```
const int mod = 1e9+7;
namespace min25{
int p[N], tot, prim[N];
int sp1[N]; // 前i个质数和
int sp2[N]; // 前i个质数平方和
// f需要是个多项式，将多项式每一项拆开做，这拆开成f1,f2
// f1, f2传质数时合法
int f1(int x) {return x;}
int f2(int x) {x%=mod; return x*x%mod;}
void init(){
    p[1]=1;
    For(i,2,N-1){
        if(!p[i]){
            p[i]=prim[++tot]=i;
            sp1[tot] = (sp1[tot-1] + f1(i)) % mod;
            sp2[tot] = (sp2[tot-1] + f2(i)) % mod;
        }
        for(int j = 1; j <= tot && prim[j]*i < N; j++){
            p[prim[j]*i] = prim[j];
            if(p[i]==prim[j]) break;
        }
    }
}
int cnt, w[N], id1[N], id2[N];
// w存所有的n/i  id1,id2存n/i的下标
int g1[N], g2[N], SQ, n;
// g滚的是第二维
// 预处理完g1后 g1[i]表示<=w[i]的所有质数的f1和，(除去1)
int f(int x){   // x是素数幂 f值
    x%=mod;
    return (x*x%mod-x+mod)%mod;
}
int wFsum(int x){
    // 定义看题目 这里要求的f是f(pp) = pp^2-pp  (pp是素数幂)
    // 表示<=w[x]的质数f和
    return (g2[x]-g1[x]+mod)%mod;
}
int pFsum(int x){        // primFsum
    // 定义看题目，表示前x个质数的f和
    return (sp2[x]-sp1[x]+mod)%mod;
}
```

```
int getid(int x) { // w和id转换
    return x<=SQ?id1[x]:id2[n/x];
}
int S(int x, int y) {    // 表示2-x，最小质因子>prim[y]的答案
    if(prim[y]>=x)return 0; // 边界情况
    // 题解说不用记忆化（不懂,不管）
    int k = getid(x);
    int res = (wFsum(k) - pFsum(y)+mod)%mod;
    for(int i = y+1; i <= tot && prim[i]*prim[i] <= x; i++){
        int pe = prim[i];
        for(int e = 1; pe <= x; e++, pe=pe*prim[i]){
            res = (res + f(pe)*(S(x/pe, i) + (e!=1))%mod)%mod;
        }
    }
    return res;
}
void calcG(){
    static int inv6 = ksm(6, mod-2);
    auto sum1 = [](int x) {
        x %= mod;
        return x*(x+1)/2%mod;
    };
    auto sum2 = [](int x){
        x %= mod;
        return x*(x+1)%mod*(2*x+1)%mod*inv6%mod;
    };
    for(int l = 1, r; l <= n; l=r+1){
        int d = n/l;
        r = n/d;
        w[++cnt] = d;
        if(d <= SQ)id1[d] = cnt;
        else id2[n/d] = cnt;
        g1[cnt] = (sum1(d)-1+mod)%mod;  // 在筛之前，他就是2~w[cnt]的前缀和
        g2[cnt] = (sum2(d)-1+mod)%mod;
    }
    for(int i = 1; i <= tot; i++){  // 枚举所有质数
        // w大小是倒着的，所以正着滚g[j]会从后面的j转移过来
        for(int j = 1; j <= cnt && prim[i]*prim[i] <= w[j]; j++){
            int k = getid(w[j]/prim[i]);
            g1[j] -= f1(prim[i]) * (g1[k] - sp1[i-1])%mod;
            g2[j] -= f2(prim[i]) * (g2[k] - sp2[i-1])%mod;
            g1[j] = (g1[j]%mod+mod)%mod;
            g2[j] = (g2[j]%mod+mod)%mod;
        }
    }
}
}
void solve()
{
```

```
    min25::init();  // 线性筛部分
    int n;
    cin >> n;
    if(n==1){
        cout << "1\n";
        return;
    }
    min25::n = n;
    min25::SQ = sqrt(n);
    min25::calcG();              // 第一步算g(质数的情况)
    int ans = min25::S(n, 0);    // 第二部算答案
    ans = (ans + 1)%mod;         // 算S时没有算f(1)
    cout << ans << "\n";
}
```

莫比乌斯反演

给两个数 n, m，求有多少对 (i, j) 满足 $1 \le i \le n, 1 \le j \le m$ 并且 $gcd(i, j) = 1$。
先设 $n \le m$ 有式子
$\sum_{i=1}^{n} \sum_{j=1}^{m} [(i, j) = 1]$
$\sum_{i=1}^{n} \sum_{j=1}^{m} e((i, j))$
$\sum_{i=1}^{n} \sum_{j=1}^{m} \sum_{d|(i,j)} \mu(d)$
$\sum_{d=1}^{n} \mu(d) \times \sum_{d|i} \sum_{d|j} 1$
$\sum_{d=1}^{n} \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
μ前缀和+整除分块可以在$\sqrt{n}$内解决

原根/阶

- 阶：因此满足同余式 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 存在，这个 n 称作 a 模 m 的阶，记作 $\delta_m(a)$ 或 $\mathrm{ord}_m(a)$.
- 原根：把阶刚好等于$\varphi(m)$的余数（同余类）称为**模m的原根(primitive root)**。
  - $m = 5$，$\varphi(m) = 4$，$2, 3$的阶都是4，$2, 3$是模5的原根。
- m的原根个数为$\varphi(\varphi(m))$
- 最小原根不大，可以暴力找，素数p的最小原根$O(m^{0.25})$

求原根

```
ll p;
cin >> p;
vector<int> v;
int phi = p-1;
int x = phi;
for(int i = 2; i*i <= x; i++)if(x%i==0){
```

```
    v.push_back(i);
    while(x%i==0) x/=i;
}
if(x!=1)v.push_back(x);
for(int g = 1; g < p; g++){
    int flag=1;
    for(auto c:v){
        if(ksm(g,phi/c,p)==1){
            flag=0;break;
        }
    }
    if(flag){
        cout << g << "\n";
        return;
    }
}
```

二次剩余(取模意义下开根)

使用欧拉准则判断有无解

满足式子 $x^2 \equiv a \mod p$ ， a 的个数有 $O(\frac{p+1}{2})$ 个

当 $p \mod 4 = 3$ 时 $x = a^{\frac{p+1}{4}} \mod p$ 为上式子的一个解

```
namespace sqrt_mod{
    ll W,W0;
    const ll mod = 1000000007;  // 模数
    struct comp{
        ll x,y;
        comp(ll xx=0,ll yy=0){x=xx,y=yy;}
    };
    comp operator * (comp a,comp b){
        return comp((a.x*b.x%mod+a.y*b.y%mod*W%mod)%mod,
                    (a.y*b.x%mod+a.x*b.y%mod)%mod);
    }
    inline ll ksm(ll a, ll b){
        ll t=1;a%=mod;
        while (b){
            if (b&1) t=t*a%mod;
            b>>=1;
            a=a*a%mod;
        }
        return t;
    }
    inline comp ksm(comp a, ll b){
        comp t=comp(1,0);
```

```
        while (b){
            if (b&1) t=t*a;
            b>>=1;a=a*a;
        }
        return t;
    }
    int legendre(int n) { // 无解 欧拉准则
        if(!(n % mod)) return 0;
        return ksm(n, (mod - 1) / 2);
    }
    // O(log(mod));
    inline pair<ll,ll> Cipolla(ll x){   // y^2 = x % mod
        // 无解 欧拉准则
        // if (ksm(x,(mod-1)>>1)==mod-1) return make_pair(-1,-1);
        if(legendre(x)==-1)return make_pair(-1, -1);    // 与上一行等价
        srand(time(NULL));
        if(x==0)return make_pair(0, 0);
        while (1){
            W0=rand()%mod;W=(W0*W0%mod-x+mod)%mod;
            if (ksm(W,(mod-1)>>1)==mod-1) break;
        }
        comp a=ksm(comp(W0,1),(mod+1)>>1);
        return make_pair(min(a.x,mod-a.x),max(a.x,mod-a.x));    // 互为相反数
    }
}
```

Pollard-rho, miller_rabin

```
int maxfac;
ll ksm(ll a, ll b, ll m){
    ll res = 1;
    a %= m;
    while(b) {
        if (b & 1) res = (__int128)res * a % m;
        b >>= 1;
        a = (__int128)a * a % m;
    }
    return res;
}
bool miller_rabin(ll p) // 判质数
{
    if (p < 2) return 0;
    if (p <= 3) return 1;
    ll d = p - 1, r = 0;
    while (!(d & 1)) ++r, d >>= 1;  // 将d处理为奇数
    for (ll k = 0; k < 10; ++k) {
        ll a = rand() % (p - 2) + 2;
        ll x = ksm(a, d, p);
```

```
        if (x == 1 || x == p - 1) continue;
        for (int i = 0; i < r - 1; ++i) {
            x = (__int128)x * x % p;
            if (x == p - 1) break;
        }
        if (x != p - 1) return 0;
    }
    return 1;
}
ll Pollard_Rho(ll x) {  // 找x的一个因子
    ll s = 0, t = 0;
    ll c = (ll)rand() % (x - 1) + 1;
    int step = 0, goal = 1;
    ll val = 1;
    for (goal = 1;; goal *= 2, s = t, val = 1){  // 倍增优化
        for (step = 1; step <= goal; ++step){
            t = ((__int128)t * t + c) % x;
            val = (__int128)val * abs(t - s) % x;
            if ((step % 127) == 0) {
                ll d = gcd(val, x);
                if (d > 1) return d;
            }
        }
        ll d = gcd(val, x);
        if (d > 1) return d;
    }
}
void findfac(ll n){
    if(n==1 || n<=maxfac)return;    // 剪枝
    if(miller_rabin(n)){    // 质数
        maxfac = max(maxfac, n);    // 找最大质因子
        return;
    }
    ll m=n;
    while(m==n)m=Pollard_Rho(n);
    while(n%m==0)n/=m;
    findfac(m), findfac(n);
}
```

矩阵

```
const int M = 4;
struct Matrix{
    int a[M][M];
    Matrix(){memset(a,0,sizeof(a));}
    Matrix(int x){
```

```
        memset(a,0,sizeof(a));
        For(i,0,M-1)a[i][i]=x;
    }
    void unit(){
        memset(a,0,sizeof(a));
        For(i,0,M-1)a[i][i]=1;
    }
    void A(){

    }
    void a0(){

    }
    Matrix operator * (Matrix& o){
        Matrix res;
        For(i,0,M-1)For(j,0,M-1)if(a[i][j])
            For(k,0,M-1){
                res.a[i][k] += a[i][j]*o.a[j][k];
            }
        return res;
    }
    void print(){
        For(i,0,M-1){
            For(j,0,M-1)cout << a[i][j] << ' ';
            cout << '\n';
        }
    }
};
```

线性基

```
const int B = 63;   // 最高位数
struct linear_basis
{
    ll num[B+1];
    bool insert(ll x)   // 将x插进线性基 true插入成功
    {
        for(int i = B; i >= 0; i--) if(x & (1ll<<i)) {
            if(num[i] == 0){
                num[i] = x;
                return true;
            }
            x ^= num[i];
        }
        return false;
    }
```

```
    ll querymin(ll x)    // 如果x为0 线性基可以表示x
    {
        for(int i = B; i >= 0; i--) {
            x = min(x, x ^ num[i]);
        }
        return x;
    }
};
```

高斯消元

```
int a[N][N];     // 增广矩阵
int ans[N], n;
void gaosi()
{
    int r = 1;
    For(i,1,n){//解第i个未知数
        int p = r;
        For(j,r,n)  // 找到一个这一项系数不为0的式子
            if(fabs(a[j][i])>fabs(a[p][i])){
                p = j; break;
            }
        if(fabs(a[p][i])<1e-6)continue;
        For(j,1,n+1)     // 把式子换到第r行
            swap(a[p][j],a[r][j]);
        Rep(j,n+1,i)     // 将a[r][i]变成1
            a[r][j] /= a[r][i];
        // 把其他式子的第i项消掉
        For(j,i+1,n){
            double cha = a[j][i];
            For(k,i,n+1)
                a[j][k] -= cha*a[r][k];
        }
        r++;
    }
    ans[n] = a[n][n+1];
    Rep(i,n-1,1){
        double res = a[i][n+1];
        For(j,i+1,n)
            res -= ans[j]*a[i][j];
        ans[i] = res;
    }
    if(r==n+1)
        For(i,1,n){
            if(fabs(ans[i])<1e-7)ans[i]=0;//防-0.00
            cout << ans[i] << "\n";
        }
    else cout << "No Solution";
```

```
}
```

拉格朗日插值

```
// n+1 个点可以唯一确定一个n次多项式
// 已知n个点(x[i], y[i])
// 求f(xx)   O(n^2)
int Lagrange(int n, int *x, int *y, int k)
{
    int ans = k;
    For(i,1,n){
        int zi=y[i], mu=1;
        For(j,1,n) if(i!=j){
            zi = zi*(k-x[j]+mod)%mod;
            mu = mu*(x[i]-x[j]+mod)%mod;
        }
        ans = (ans + zi*ksm(mu, mod-2)%mod)%mod;
    }
    return ans;
}

// 当x取值为连续整数时， 可以优化到O(n)
int pre[N], inv[N]; // (k-x[i])的前缀积和 1/(k-x[i])
int jie[N], ijie[N];
int Lagrange_On(int n, int *x, int *y, int k)
{
    // 要保证 k > x[n] (x要满足 x[i+1] - x[i] = 1)
    int ans = 0;
    pre[0]=1;
    For(i,1,n)pre[i] = pre[i-1]*(k-x[i])%mod;
    inv[n] = ksm(pre[n], mod-2);
    Rep(i,n,1){
        inv[i-1] = inv[i]*(k-x[i])%mod;
        inv[i] = inv[i]*pre[i-1]%mod;
    }
    For(i,1,n){
        int num = ijie[i-1] * ijie[n-i]%mod * inv[i]%mod * pre[n]%mod;
        int f = ((n-i)&1?-1:1);
        ans = (ans + f*num*y[i]%mod)%mod;
    }
    return ans<0?ans+mod:ans;
}
```

康托展开(求一个排列是字典序第几个)

给定排列$p_1, p_2, ... p_n$。比他小的排列有:

$$\sum_{i=1}^{n} num_i \times (n-i)!$$

其中num_i表示i后面比p_i小的元素个数。当前排列在全排列的排名就是上面式子$+1$

快速沃尔什变换(FWT)

$$c_i = \sum_{i=j\oplus k} a_j b_k$$

$\oplus$为异或，或，与三种运算中的一种，如果只需要一种运算，抄的时候可以将FWT和UFWT合并

```
/**
 * 不需要取模就将mod去掉
*/
void FWT(int* a, int n){
    for(int d = 1; d < n; d <<= 1){
        for(int m = (d<<1), i = 0; i < n; i += m){
            for(int j = 0; j < d; j++){
                int x = a[i+j], y = a[i+j+d];
                //xor: a[i+j] = x+y, a[i+j+d] = x-y;
                //and: a[i+j] = x+y;
                //or:  a[i+j+d] = x+y;
            }
        }
    }
}
void UFWT(int* a, int n){
    // 这的3个for和上面完全一样
    for(int d = 1; d < n; d <<= 1){
        for(int m = (d<<1), i = 0; i < n; i += m){
            for(int j = 0; j < d; j++){
                int x = a[i+j], y = a[i+j+d];
                //xor: a[i+j] = (x+y)/2, a[i+j+d] = (x-y)/2;
                //and: a[i+j] = x-y;
                //or:  a[i+j+d] = y-x;
            }
        }
    }
}
// a · b -> c, a,b长度都为n 下标为[0, n-1]
void mul(int* a, int* b, int* c, int n)
{
    FWT(a, n), FWT(b, n);
    For(i,0,n-1) c[i] = a[i]*b[i]%mod;
    UFWT(c, n);
}
```

子集卷积

$$c_k = \sum_{\substack{i\&j=0 \\ i\,|\,j=k}} a_i b_j$$

$i\&j$转化为$|i|+|j|=|i\vee j|$，开多一维记录元素个数，FWT开，然后有

$$c_{i,j} = \sum_{k=0}^{i} a_{k,j} b_{i-k,j}$$

然后FWT回去，原题目的c_i就是$c_{popcount(i),i}$

```
    int a[21][1<<20], b[21][1<<20];
    cin >> n;
    m = (1<<n);
    For(i,0,m-1)cin >> a[__builtin_popcount(i)][i];
    For(i,0,m-1)cin >> b[__builtin_popcount(i)][i];
    For(i,0,n){
        FWT(a[i],m,0);
        FWT(b[i],m,0);
    }
    For(i,0,n){
        For(j,0,m-1){
            For(k,0,i){
                c[i][j] = norm(c[i][j] + a[k][j]*b[i-k][j]%mod);
            }
        }
    }
    For(i,0,n)FWT(c[i], m, 1);
    For(i,0,m-1){
        cout << c[__builtin_popcount(i)][i] << " \n"[i==m-1];
    }
```

数列表

第二类斯特林数(mod 998244353)

```
1
1 1
1 3 1
1 7 6 1
1 15 25 10 1
1 31 90 65 15 1
1 63 301 350 140 21 1
1 127 966 1701 1050 266 28 1
```

```
1 255 3025 7770 6951 2646 462 36 1
1 511 9330 34105 42525 22827 5880 750 45 1
1 1023 28501 145750 246730 179487 63987 11880 1155 55 1
1 2047 86526 611501 1379400 1323652 627396 159027 22275 1705 66 1
1 4095 261625 2532530 7508501 9321312 5715424 1899612 359502 39325 2431 78 1
1 8191 788970 10391745 40075035 63436373 49329280 20912320 5135130 752752 66066 3367 91 1
```

卡特兰数(mod 998244353)

```
1 1 2 5 14 42 132 429 1430 4862 16796 58786 208012 742900 2674440 9694845 35357670 129644790 477638700 769018837 574654302
```

多项式

FFT

- 精度保证：系数总和 $\leq 10^{15}$ 开`long duoble`
- 精度够(maybe总和$\leq 10^{10}$)可以不开`long double`
- 先调用`fft_init()`

```
// FFT_MAXN = 2^k
// fft_init() to precalc FFT_MAXN-th roots
typedef long double db;
const int FFT_MX=262144, N = 301000;
const db pi=acosl(-1.);
struct cp{
    db a,b;
    cp operator+(const cp&y)const{return (cp){a+y.a,b+y.b};}
    cp operator-(const cp&y)const{return (cp){a-y.a,b-y.b};}
    cp operator*(const cp&y)const{return (cp){a*y.a-b*y.b,a*y.b+b*y.a};}
    cp operator!()const{return (cp){a,-b};};
}nw[FFT_MX+1];int bitrev[FFT_MX];

void dft(cp*a,int n,int flag=1){
    int d=0; while((1<<d)*n!=FFT_MX) d++;
    For(i,0,n-1) if(i < (bitrev[i]>>d))
        swap(a[i], a[bitrev[i]>>d]);
    for (int l=2;l<=n;l<<=1){
        int del=FFT_MX/l*flag;
        for (int i=0;i<n;i+=l){
            cp *le=a+i, *ri=a+i+(l>>1), *w=(flag==1) ? nw : nw+FFT_MX;
            For(k,0,(l>>1)-1){
                cp ne=*ri**w;
                *ri=*le-ne, *le=*le+ne;
                le++, ri++, w+=del;
            }
        }
    }
    if(flag!=1) For(i,0,n-1)
        a[i].a/=n, a[i].b/=n;
}
void fft_init(){
    int L=0;
    while((1<<L)!=FFT_MX) L++;
    bitrev[0]=0;
    For(i,1,FFT_MX-1)
        bitrev[i] = bitrev[i>>1]>>1 | ((i&1)<<(L-1));
```

```
    nw[0]=nw[FFT_MX]=(cp){1,0};
    //very slow
    For(i,0,FFT_MX)nw[i] = (cp){cosl(2*pi/FFT_MX*i), sinl(2*pi/FFT_MX*i)};
}
// n, m 分别为a, b的最高次幂, 数组a的范围为[0, n], b为[0, m], c转整数四舍五入
void polymul(db *a, int n, db *b, int m, db *c) {
    static cp f[FFT_MX>>1], g[FFT_MX>>1], t[FFT_MX>>1];
    int N=2;
    while(N<=n+m)N<<=1;
    For(i,0,N-1)     // 此N非全局的N
        if(i&1){
            f[i>>1].b=(i<=n)?a[i]:0.0;
            g[i>>1].b=(i<=m)?b[i]:0.0;
        }else{
            f[i>>1].a=(i<=n)?a[i]:0.0;
            g[i>>1].a=(i<=m)?b[i]:0.0;
        }
    dft(f,N>>1); dft(g,N>>1);
    int del=FFT_MX/(N>>1);
    cp qua=(cp){0,0.25}, one=(cp){1,0}, four=(cp){4,0}, *w=nw;
    For(i,0,(N>>1)-1){
        int j=i?(N>>1)-i:0;
        t[i] = (four*!(f[j]*g[j])-(!f[j]-f[i])*(!g[j]-g[i])*(one+*w))*qua;
        w+=del;
    }
    dft(t,N>>1,-1);
    For(i,0,n+m)c[i]=(i&1) ? t[i>>1].a : t[i>>1].b;
}
// 这实际抄的时候可以和上面结合一下 少个复制的常数
db A[N], B[N], C[N];
vector<ll> vectorMul(const vector<ll>& a, const vector<ll>& b){
    int n = a.size(), m = b.size();
    if(n==0 || m==0)return {};
    n--, m--;
    For(i,0,n)A[i] = a[i];
    For(i,0,m)B[i] = b[i];
    polymul(A, n, B, m, C);
    vector<ll> res(n+m+1);
    For(i,0,n+m) res[i] = C[i]+0.5;
    return res;
}
```

NTT

```
int g=3;//mod的原根
int gi=ksm(g,mod-2); //g 的逆元
int rev[N];
ll a[2*N],b[2*N];
```

```cpp
void ntt(ll A[], int tot, int type){
    for(int i = 0; i < tot; i++)
        if(i<rev[i]) swap(A[i], A[rev[i]]);
    for(int mid = 1; mid < tot; mid <<= 1){
        ll gn = ksm(type>0 ? g : gi, (mod-1)/(mid<<1));//与下面ifelse等价 可整除
        // if(type==1)gn = ksm(g, (mod-1)/(mid<<1));
        // else gn = ksm(gi, (mod-1)/(mod<<1));
        int R = mid<<1;
        for(int i = 0; i < tot; i+=R){
            ll g0 = 1;
            for(int j = i; j < mid+i; j++, g0=g0*gn%mod){
                ll x = A[j], y=g0*A[j+mid]%mod;
                A[j] = (x+y)%mod;
                A[j+mid] = (x-y+mod)%mod;
            }
        }
    }
    ll invtot = ksm(tot, mod-2);
    if(type==-1){
        for(int i = 0; i < tot; i++)
            A[i] = A[i]*invtot%mod;
    }
}
// a=a*b
void polymul_ntt(ll a[], ll b[],int n,int m){
    tot=1;
    int l=0;    //求rev
    while(tot <= n+m){
        tot<<=1; l++;
    }
    for(int i = 0; i <= tot; i++)
        rev[i] = (rev[i>>1]>>1) | ((i&1)<<(l-1));
    ntt(a,tot,1);
    ntt(b,tot,1);
    for(int i = 0; i <= tot; i++)
        a[i] = a[i]*b[i]%mod;
    ntt(a,tot,-1);
}
```

多项式求逆

```cpp
// 最高次项为len-1 的多项式f  返回多项式的逆g  倍增法求解  需要2个辅助数组c d
ll c[N],d[N];
void polyinv(int len,ll f[], ll g[]) {
    if(len==1){
        g[0]=ksm(f[0],mod-2);
        return;
```

```
    }
    polyinv((len+1)>>1,f,g);
    int tot=1,l=0;
    while(tot<=len+n){
        tot<<=1;l++;
    }
    For(i,0,tot-1)rev[i] = (rev[i>>1]>>1) | ((i&1)<<(l-1));
    For(i,0,tot-1){
        c[i]=f[i];
        if(i<((len+1)>>1))d[i]=g[i];
        else d[i]=0;
    }
    ntt(c,1,tot);
    ntt(d,1,tot);
    For(i,0,tot-1){
        c[i] = (d[i]*((2-c[i]*d[i]%mod+mod)%mod)%mod)%mod;
    }
    ntt(c,-1,tot);  //ntt里已经除了tot
    for(int i = 0; i < len; i++)
        g[i] = c[i];
}
```

NTT(vector)

```
const int md = 998244353;

inline void add(int &x, int y) {
  x += y;
  if (x >= md) {
    x -= md;
  }
}

inline void sub(int &x, int y) {
  x -= y;
  if (x < 0) {
    x += md;
  }
}

inline int mul(int x, int y) {
  return (long long) x * y % md;
}

inline int power(int x, int y) {
  int res = 1;
```

```
  for (; y; y >>= 1, x = mul(x, x)) {
    if (y & 1) {
      res = mul(res, x);
    }
  }
  return res;
}

inline int inv(int a) {
  a %= md;
  if (a < 0) {
    a += md;
  }
  int b = md, u = 0, v = 1;
  while (a) {
    int t = b / a;
    b -= t * a;
    swap(a, b);
    u -= t * v;
    swap(u, v);
  }
  if (u < 0) {
    u += md;
  }
  return u;
}

namespace ntt {
int base = 1, root = -1, max_base = -1;
vector<int> rev = {0, 1}, roots = {0, 1};

void init() {
  int temp = md - 1;
  max_base = 0;
  while (temp % 2 == 0) {
    temp >>= 1;
    ++max_base;
  }
  root = 2;
  while (true) {
    if (power(root, 1<<max_base)==1 && power(root, 1<<(max_base-1))!=1){
      break;
    }
    ++root;
  }
}

void ensure_base(int nbase) {
  if (max_base == -1) {
```

```
    init();
  }
  if (nbase <= base) {
    return;
  }
  assert(nbase <= max_base);
  rev.resize(1 << nbase);
  for (int i = 0; i < 1 << nbase; ++i) {
    rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (nbase - 1));
  }
  roots.resize(1 << nbase);
  while (base < nbase) {
    int z = power(root, 1 << (max_base - 1 - base));
    for (int i = 1 << (base - 1); i < 1 << base; ++i) {
      roots[i << 1] = roots[i];
      roots[i << 1 | 1] = mul(roots[i], z);
    }
    ++base;
  }
}

void dft(vector<int> &a) {
  int n = a.size(), zeros = __builtin_ctz(n);
  ensure_base(zeros);
  int shift = base - zeros;
  for (int i = 0; i < n; ++i) {
    if (i < rev[i] >> shift) {
      swap(a[i], a[rev[i] >> shift]);
    }
  }
  for (int i = 1; i < n; i <<= 1) {
    for (int j = 0; j < n; j += i << 1) {
      for (int k = 0; k < i; ++k) {
        int x = a[j + k], y = mul(a[j + k + i], roots[i + k]);
        a[j + k] = (x + y) % md;
        a[j + k + i] = (x + md - y) % md;
      }
    }
  }
}

vector<int> multiply(vector<int> a, vector<int> b) {
  int need = a.size() + b.size() - 1, nbase = 0;
  while (1 << nbase < need) {
    ++nbase;
  }
  ensure_base(nbase);
  int sz = 1 << nbase;
  a.resize(sz);
```

```
  b.resize(sz);
  bool equal = a == b;
  dft(a);
  if (equal) {
    b = a;
  } else {
    dft(b);
  }
  int inv_sz = inv(sz);
  for (int i = 0; i < sz; ++i) {
    a[i] = mul(mul(a[i], b[i]), inv_sz);
  }
  reverse(a.begin() + 1, a.end());
  dft(a);
  a.resize(need);
  return a;
}

vector<int> inverse(vector<int> a) {
  int n = a.size(), m = (n + 1) >> 1;
  if (n == 1) {
    return vector<int>(1, inv(a[0]));
  } else {
    vector<int> b = inverse(vector<int>(a.begin(), a.begin() + m));
    int need = n << 1, nbase = 0;
    while (1 << nbase < need) {
      ++nbase;
    }
    ensure_base(nbase);
    int sz = 1 << nbase;
    a.resize(sz);
    b.resize(sz);
    dft(a);
    dft(b);
    int inv_sz = inv(sz);
    for (int i = 0; i < sz; ++i) {
      a[i] = mul(mul(md + 2 - mul(a[i], b[i]), b[i]), inv_sz);
    }
    reverse(a.begin() + 1, a.end());
    dft(a);
    a.resize(n);
    return a;
  }
}
}

using ntt::multiply;
using ntt::inverse;

```

```
vector<int>& operator += (vector<int> &a, const vector<int> &b) {
  if (a.size() < b.size()) {
    a.resize(b.size());
  }
  for (int i = 0; i < b.size(); ++i) {
    add(a[i], b[i]);
  }
  return a;
}

vector<int> operator + (const vector<int> &a, const vector<int> &b) {
  vector<int> c = a;
  return c += b;
}

vector<int>& operator -= (vector<int> &a, const vector<int> &b) {
  if (a.size() < b.size()) {
    a.resize(b.size());
  }
  for (int i = 0; i < b.size(); ++i) {
    sub(a[i], b[i]);
  }
  return a;
}

vector<int> operator - (const vector<int> &a, const vector<int> &b) {
  vector<int> c = a;
  return c -= b;
}

vector<int>& operator *= (vector<int> &a, const vector<int> &b) {
  if (min(a.size(), b.size()) < 128) {
    vector<int> c = a;
    a.assign(a.size() + b.size() - 1, 0);
    for (int i = 0; i < c.size(); ++i) {
      for (int j = 0; j < b.size(); ++j) {
        add(a[i + j], mul(c[i], b[j]));
      }
    }
  } else {
    a = multiply(a, b);
  }
  return a;
}

vector<int> operator * (const vector<int> &a, const vector<int> &b) {
  vector<int> c = a;
  return c *= b;
}
```

```

vector<int>& operator /= (vector<int> &a, const vector<int> &b) {
  int n = a.size(), m = b.size();
  if (n < m) {
    a.clear();
  } else {
    vector<int> c = b;
    reverse(a.begin(), a.end());
    reverse(c.begin(), c.end());
    c.resize(n - m + 1);
    a *= inverse(c);
    a.erase(a.begin() + n - m + 1, a.end());
    reverse(a.begin(), a.end());
  }
  return a;
}

vector<int> operator / (const vector<int> &a, const vector<int> &b) {
  vector<int> c = a;
  return c /= b;
}

vector<int>& operator %= (vector<int> &a, const vector<int> &b) {
  int n = a.size(), m = b.size();
  if (n >= m) {
    vector<int> c = (a / b) * b;
    a.resize(m - 1);
    for (int i = 0; i < m - 1; ++i) {
      sub(a[i], c[i]);
    }
  }
  return a;
}

vector<int> operator % (const vector<int> &a, const vector<int> &b) {
  vector<int> c = a;
  return c %= b;
}

vector<int> derivative(const vector<int> &a) {
  int n = a.size();
  vector<int> b(n - 1);
  for (int i = 1; i < n; ++i) {
    b[i - 1] = mul(a[i], i);
  }
  return b;
}

vector<int> primitive(const vector<int> &a) {
```

```
  int n = a.size();
  vector<int> b(n + 1), invs(n + 1);
  for (int i = 1; i <= n; ++i) {
    invs[i] = i == 1 ? 1 : mul(md - md / i, invs[md % i]);
    b[i] = mul(a[i - 1], invs[i]);
  }
  return b;
}

vector<int> logarithm(const vector<int> &a) {
  vector<int> b = primitive(derivative(a) * inverse(a));
  b.resize(a.size());
  return b;
}

vector<int> exponent(const vector<int> &a) {
  vector<int> b(1, 1);
  while (b.size() < a.size()) {
    vector<int> c(a.begin(), a.begin() + min(a.size(), b.size() << 1));
    add(c[0], 1);
    vector<int> old_b = b;
    b.resize(b.size() << 1);
    c -= logarithm(b);
    c *= old_b;
    for (int i = b.size() >> 1; i < b.size(); ++i) {
      b[i] = c[i];
    }
  }
  b.resize(a.size());
  return b;
}

vector<int> power(vector<int> a, int m) {
  int n = a.size(), p = -1;
  vector<int> b(n);
  for (int i = 0; i < n; ++i) {
    if (a[i]) {
      p = i;
      break;
    }
  }
  if (p == -1) {
    b[0] = !m;
    return b;
  }
  if ((long long) m * p >= n) {
    return b;
  }
  int mu = power(a[p], m), di = inv(a[p]);
```

```
  vector<int> c(n - m * p);
  for (int i = 0; i < n - m * p; ++i) {
    c[i] = mul(a[i + p], di);
  }
  c = logarithm(c);
  for (int i = 0; i < n - m * p; ++i) {
    c[i] = mul(c[i], m);
  }
  c = exponent(c);
  for (int i = 0; i < n - m * p; ++i) {
    b[i + m * p] = mul(c[i], mu);
  }
  return b;
}

vector<int> sqrt(const vector<int> &a) {
  vector<int> b(1, 1);
  while (b.size() < a.size()) {
    vector<int> c(a.begin(), a.begin() + min(a.size(), b.size() << 1));
    vector<int> old_b = b;
    b.resize(b.size() << 1);
    c *= inverse(b);
    for (int i = b.size() >> 1; i < b.size(); ++i) {
      b[i] = mul(c[i], (md + 1) >> 1);
    }
  }
  b.resize(a.size());
  return b;
}

vector<int> multiply_all(int l, int r, vector<vector<int>> &all) {
  if (l > r) {
    return vector<int>();
  } else if (l == r) {
    return all[l];
  } else {
    int y = (l + r) >> 1;
    return multiply_all(l, y, all) * multiply_all(y + 1, r, all);
  }
}

vector<int> evaluate(const vector<int> &f, const vector<int> &x) {
  int n = x.size();
  if (!n) {
    return vector<int>();
  }
  vector<vector<int>> up(n * 2);
  for (int i = 0; i < n; ++i) {
    up[i + n] = vector<int>{(md - x[i]) % md, 1};
```

```
  }
  for (int i = n - 1; i; --i) {
    up[i] = up[i << 1] * up[i << 1 | 1];
  }
  vector<vector<int>> down(n * 2);
  down[1] = f % up[1];
  for (int i = 2; i < n * 2; ++i) {
    down[i] = down[i >> 1] % up[i];
  }
  vector<int> y(n);
  for (int i = 0; i < n; ++i) {
    y[i] = down[i + n][0];
  }
  return y;
}

vector<int> interpolate(const vector<int> &x, const vector<int> &y) {
  int n = x.size();
  vector<vector<int>> up(n * 2);
  for (int i = 0; i < n; ++i) {
    up[i + n] = vector<int>{(md - x[i]) % md, 1};
  }
  for (int i = n - 1; i; --i) {
    up[i] = up[i << 1] * up[i << 1 | 1];
  }
  vector<int> a = evaluate(derivative(up[1]), x);
  for (int i = 0; i < n; ++i) {
    a[i] = mul(y[i], inv(a[i]));
  }
  vector<vector<int>> down(n * 2);
  for (int i = 0; i < n; ++i) {
    down[i + n] = vector<int>(1, a[i]);
  }
  for (int i = n - 1; i; --i) {
    down[i] = down[i << 1] * up[i << 1 | 1] + down[i << 1 | 1] * up[i << 1];
  }
  return down[1];
}
```

图论

强连通分量

```
int dfn[N], low[N], idx, ins[N], cnt, bel[N];
stack<int> st;
void dfs(int u)
{
    dfn[u] = low[u] = ++idx;
    ins[u] = 1;
    st.push(u);
    for(auto v:G[u]){
        if(!dfn[v])dfs(v);
        if(ins[v])low[u] = min(low[u], low[v]);
    }
    if(low[u] == dfn[u]){
        cnt++;
        while(true){
            int x = st.top(); st.pop();
            ins[x]=0;
            bel[x] = cnt;
            if(x==u)break;
        }
    }
}
```

割边 桥

```
int dfn[N], low[N], idx;
vector<int> bridge;
void dfs(int u, int id) // 通过id边下来
{
    dfn[u] = low[u] = ++idx;
    for(auto [v, idd] : G[u]){
        if(!dfn[v]) dfs(v, idd);    // 儿子
        if(idd != id) low[u] = min(low[u], low[v]); // 不是id的边
    }
    if(dfn[u] == low[u] && id != 0){
        bridge.push_back(id);
        /* 可以和强连通分量一样把边双找出来 */
    }
}
```

割点

```
int dfn[N], low[N], idx, iscut[N], cnt, visE[N];
vector<int> dcc[N];       // 每个点双有哪些点（割点可能在多个点双内）
vector<pii> dccE[N];      // 每个点双有哪些边（每个点双单独建图用）
stack<int> st;
stack<pii> eST;
int rt;
void dfs(int u, int fa)
{
    if(fa==0)rt = u;
    dfn[u] = low[u] = ++idx;
    st.push(u);
    if(u == rt && G[u].size() == 0){
        cnt++;
        dcc[cnt].push_back(u);
        return;
    }
    int child = 0;  // 孩子数
    for(auto [v, idd] : G[u])if(v!=fa && !visE[idd]){
        visE[idd] = 1;  // 边的栈去重
        eST.push({u, v});
        if(!dfn[v]){
            dfs(v, u);
            child++;
            low[u] = min(low[u], low[v]);
            if(low[v] >= dfn[u]){   // 子树v跳不出去 u是割点
                iscut[u] = 1;
                cnt++;
                while(true){  // 找点双的点
                    int x = st.top(); st.pop();
                    dcc[cnt].push_back(x);
                    if(x==v)break;
                }
                dcc[cnt].push_back(u);
                while(eST.size()){  // 记边
                    auto e = eST.top(); eST.pop();
                    dccE[cnt].push_back(e);
                    if(e == make_pair(u, v) || e == make_pair(v, u))break;
                }
            }
        } else if(v != fa){
            low[u] = min(low[u], dfn[v]);   // 不能跳多步 只能dfn[v]
        }
    }
    if(u == rt && child <= 1) iscut[u] = 0;
}
```

二分图最大匹配

二分图最小路径覆盖 $=$ 最大独立集 $=$ 总节点数 $-$ 最大匹配数，最小点覆盖 $=$ 最大匹配数。

任意图中，最大独立集 $+$ 最小点覆盖集 $= V$，最大团 $=$ 补图的最大独立集。

独立集：点集，两两间没有边相连

二分图最大匹配：在图中选出一些边，使得这些边没有公共顶点，且边的数量最大

团：完全子图

最小点覆盖：选最少的点，满足每条边至少有一个端点被选。

最小路径覆盖：找出最少的路径，使得这些路径经过了所有的点。

最小路径覆盖分为最小不相交路径覆盖和最小可相交路径覆盖。

```
vector<int> G[N];
// col[i]i在左右  lef[i] 右边的节点i匹配左边节点lef[i]
int n, m, col[N], vis[N], lef[N];
void dfs(int u)
{
    for(auto v:G[u]){
        if(!col[v]){
            col[v] = 3-col[u];
            dfs(v);
        }
    }
}
bool find(int u)
{
    for(auto v:G[u]) if(!vis[v]){
        vis[v]=1;
        if(!lef[v] || find(lef[v])){
            lef[v]=u;
            return true;
        }
    }
    return false;
}
int match() // 二分图最大匹配
{
    int ans = 0;
    for(int i = 1; i <= n; i++){     // 跑二分图一边
        if(col[i]==1){
            for(int j = 1; j <= n; j++)vis[j]=0;
            if(find(i))ans++;
        }
    }
```

```
    return ans;
}
```

二分图最大权匹配

左边n个节点，右边m个节点，选择(u,v)匹配的权值为w，求一组匹配使得权值和最大。时间复杂度$\mathcal{O}(n^3)$

```
#include<bits/stdc++.h>
#define For(i,a,b) for(int i = a; i <= b; ++i)
using namespace std;
// R >= L, 左边完全匹配
// 最小权完备匹配
/**
 * @param a 图，邻接矩阵，下标0开始，行为L，列为R
 * @return pair(v,ans). v为匹配的值，
 *          ans为左边的每个点匹配右边的哪个点，不匹配是0(可能)
*/
template<class T>
pair<T, vector<int>> hungarian(const vector<vector<T>> &a) {
    if (a.empty()) return {0, {}};
    int n = a.size() + 1, m = a[0].size() + 1;
    vector<T> u(n), v(m); // 顶标
    vector<int> p(m), ans(n - 1);
    for (int i = 1; i < n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<T> dist(m, numeric_limits<T>::max());
        vector<int> pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1;
            T delta = numeric_limits<T>::max();
            for (int j = 1; j < m; j++) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            for (int j = 0; j < m; j++) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
```

```
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
    }
    for (int j = 1; j < m; j++) {
        if (p[j]) ans[p[j] - 1] = j - 1;
    }
    return {-v[0], ans}; // min cost
}
int L, R, m;
signed main()
{
    ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);
    cin >> L >> R >> m;
    R = max(L, R);  // 保证 L<= R
    vector<vector<int> > g(L, vector<int>(R, 0));
    For(i,1,m){
        int u, v, w;
        cin >> u >> v >> w;
        u--,v--;
        g[u][v] = -w;
    }
    auto [val, res] = hungarian(g);
    cout << -val << '\n';
    For(i,0,L-1){
        if(g[i][res[i]] == 0)cout << "0 ";
        else cout << res[i]+1 << ' ';
    }
    return 0;
}
```

Dilworth定理(链与反链)

在DAG范围内讨论（偏序集）

- 链：点的集合，集合中两点u, v，要么满足u能到v，要么满足v能到u.
- 反链：点的集合，集合中任意两点不可达

最小链覆盖 = 最长反链长度
最小反链覆盖 = 最长链长度

在序列中应用：问序列能划分为多少个非递增序列，可以转化为最长上升子序列问题。

最大流

- 求最大权闭合子图权值和：

- 如果一个点被选择了则后继必须被选择，那么称该图是闭合的
- **最大权闭合子图**的权值和 $\rightarrow$
  - max{被选择的点权和}
  - 正点权和-min{没被选择的正权点之和 + 被选择的负权点绝对值和}
  - 正点权和-最小割
- 建图：s和正权点连边（边权为对应点权），负权点连t（边权为点权绝对值）原来的边边权为正无穷

- 最小链(路径)覆盖，在DAG中用最少的路径覆盖所有点：
  - 路径不可相交的情况：拆点u, u'，对于边$u \rightarrow v$，建边$u - v'$，最少路径数就是$n -$ 最大匹配数
  - 路径可相交的情况：先用floyd求传递闭包，然后转化为不可相交的情况做
- 有源汇上下界最小费用可行流
  1. 附加点SS, TT
  2. 对于原图的每个点(包括s, t)。$d[u]$为u的入边流量**下界**－出边流量**下界**
     1. $d[u] < 0$：连边`(u, TT, -d[u], 0)`
     2. $d[u] > 0$：连边`(SS, u, d[u], 0)`
  3. 对于原图的边$u \rightarrow v$下界为l，上界为r，代价为w：连边`(u, v, r-l, w)`
  4. 连边`(t, s, inf, 0)`，将原图变为循环流
  5. 最后SS, TT跑MCMF出来的费用，加上原图每条边的下界乘费用。就是所求。
- 有源汇上下界最大流
  - 使用可行流的建图方法找到一个可行流（源为SS，汇为TT）
  - 然后将源汇换回s, t，删除边$t \rightarrow s$，在残量网络上跑最大流(将剩余流量榨干)
  - 两次跑出来的流量相加即为答案
- 有源汇上下界最小流
  - 和上下界最大流类似，在残量网络上跑源点为t，汇点为s的最大流（退掉不必要的流量）
  - 两次的答案相减

最坏复杂度为$O(n^2m)$，n为点数，m为边数（一般卡不满）

```
const int V = 2e4+100;
const int E = 5e4+100;
struct FlowGraph{
    int s, t, vtot, etot;
    int head[V], dis[V], cur[V];
    int d[V];   // 上下界网络流用
    struct edge{
        int v, nxt;
        ll cap;
    }e[E<<1];
    void add(int u, int v, ll f, ll ff = 0){
        e[etot] = {v, head[u], f}; head[u] = etot++;
        e[etot] = {u, head[v], ff}; head[v] = etot++;
```

```
    }
    void addBound(int u, int v, int l, int r){  // 加个带下界的边
        assert(l<=r);
        add(u, v, r-l);
        d[v] += l, d[u] -= l;
    }
    void resolveBound(){
        For(i,1,vtot){
            if(d[i]>0)add(s, i, d[i]);
            else if(d[i]<0)add(i, t, -d[i]);
        }
    }
    void init(int _s, int _t, int _vtot){
        s = _s, t = _t, vtot = _vtot;
        etot = 0;
        For(i,1,vtot)head[i] = -1;
    }
    bool bfs()
    {
        For(i,1,vtot){
            dis[i] = 0;
            cur[i] = head[i];
        }
        queue<int> q;
        q.push(s); dis[s] = 1;
        while(q.size()){
            int u = q.front(); q.pop();
            for(int i = head[u]; ~i; i = e[i].nxt){
                if(e[i].cap && !dis[e[i].v]){
                    int v = e[i].v;
                    dis[v] = dis[u] + 1;
                    if(v == t)return true;
                    q.push(v);
                }
            }
        }
        return false;
    }
    ll dfs(int u, ll mx)
    {
        if(u == t)return mx;
        ll flow = 0;
        for(int i = cur[u]; ~i; cur[u] = i = e[i].nxt){
            if(e[i].cap && dis[e[i].v] == dis[u] + 1){
                int f = dfs(e[i].v, min(mx, e[i].cap));
                e[i].cap -= f;
                e[i^1].cap += f;
                mx -= f;
                flow += f;
```

```
                if(!mx)break;
            }
        }
        if(!flow)dis[u] = -1;
        return flow;
    }
    ll dinic()
    {
        int flow = 0;
        while(bfs())
            flow += dfs(s, numeric_limits<ll>::max());
        return flow;
    }
};
```

最小费用最大流(MCMF)

最坏时间复杂度为$O(nmf)$，n为点数，m为边数，f为最大流

```
const int V = 410;
const int E = 15010;
struct MinCostFlow{
    int s, t, vtot, etot;
    int head[V];
    int pre[V];     // 记路径
    ll dis[V], flow, cost;
    bool inq[V];
    struct edge{
        int v, nxt;
        ll f, c;
    }e[E<<1];
    void add(int u, int v, ll f, ll c, ll ff = 0){
        e[etot] = {v, head[u], f, c}; head[u] = etot++;
        e[etot] = {u, head[v], ff, -c}; head[v] = etot++;
    }
    void init(int _s, int _t, int _vtot){
        s = _s, t = _t, vtot = _vtot;
        etot = 0;
        For(i,0,vtot)head[i] = -1;
    }
    bool spfa(){
        ll inf = numeric_limits<ll>::max()/2;// ll最大值/2
        For(i,1,vtot){
            dis[i] = inf;
            inq[i] = false;
            pre[i] = -1;
        }
        dis[s] = 0;
```

```
        inq[s] = true;
        queue<int> q;
        q.push(s);
        while(!q.empty()){
            int u = q.front();q.pop();
            inq[u] = false;
            for(int i = head[u]; ~i; i = e[i].nxt){
                int v = e[i].v;
                if(e[i].f && dis[v] > dis[u]+e[i].c){
                    dis[v] = dis[u]+e[i].c;
                    pre[v] = i;
                    if(!inq[v]){
                        // if(++cnt[v] >= vtot)->有负环     // cnt[]为松弛次数
                        inq[v] = true;
                        q.push(v);
                    }
                }
            }
        }
        // return dis[t] < 0;    // 没有下界最小费用可行流
        return dis[t] != inf;
    }
    void augment(){
        int u = t;
        ll f = numeric_limits<ll>::max();
        while(~pre[u]){ // 找路径上最小值
            f = min(f, e[pre[u]].f);
            u = e[pre[u]^1].v;
        }
        flow += f;
        cost += f*dis[t];
        u = t;
        while(~pre[u]){      // 路劲上减掉流的(反向边加上)
            e[pre[u]].f -= f;
            e[pre[u]^1].f += f;
            u = e[pre[u]^1].v;
        }
    }
    pii solve(){
        flow = cost = 0;
        while(spfa())augment();
        return make_pair(flow, cost);
    }
};
```

一般图最大匹配(带花树)

时间复杂度为$O(nm^2)$

和二分图的区别就是有无奇环，先和匈牙利差不多，对每个未匹配的点找增广路(bfs)，遇到奇环(u,v)两点颜色一样，u,v往上走到lca，将环上的点都缩起来(缩花/开花)，代码中没有真的缩起来，用并查集记点在哪朵花中。

$lca(u,v)$有一个性质：lca往前的那条边是匹配的，往环方向的那两条边是未匹配的。

```
// 带花树算法，求一般图最大匹配 O(n m^2) 不满
// 洛谷 n=1000 m=5e4   < 50ms
int p[N];   // 并查集
int find(int x){return x==p[x]?x:p[x]=find(p[x]);}
int match[N];   // 和 i 匹配的点
int pre[N], col[N]; // i的前驱, i的颜色,
int tim, dfn[N];    // tim要全局，或者清空dfn(只是记个时间, 不是真的dfn)
bool augment(int s) // 找增广路
{
    For(i,1,n){
        pre[i] = col[i] = 0;
        p[i] = i;
    }
    queue<int> q;   // 只有白色点才会入队

    auto LCA = [&](int x, int y){
        ++tim;
        x = find(x), y = find(y);   // 因为带了并查集，每个点均摊O(1)
        while(dfn[x] != tim){   // 两个点都经过的点就是lca
            dfn[x] = tim;
            x = find(pre[match[x]]);// lca一定是白的，可以隔着跳
            if(y)swap(x, y);    // 轮流跳
        }
        return x;
    };

    auto blossom = [&](int x, int y, int lc){
        while(find(x) != lc){
            pre[x] = y, y = match[x];
            if(col[y] == 2){
                col[y] = 1;
                q.push(y);  // 染黑入队
            }
            if(find(x) == x) p[x] = lc;
            if(find(y) == y) p[y] = lc;
            x = pre[y];
        }
    };

    q.push(s);
    col[s] = 1; // 1为白色 2为黑色
    while(q.size()) {   // bfs
        int u = q.front(); q.pop();
```

```
        for(int v:G[u]){
            if(find(v) == find(u) || col[v]==2)continue;
            if(!col[v]){    // v未被染色
                pre[v] = u, col[v] = 2;
                if(!match[v]){// v没被匹配, 找到一条增广路(s->v)
                    while(v){
                        // 不能用pre[pre[v]], 在开花改乱了pre
                        int ppre = match[pre[v]];
                        match[v] = pre[v];
                        match[pre[v]] = v;
                        v = ppre;
                    }
                    return true;
                }
                col[match[v]] = 1;
                q.push(match[v]);
            }
            else{   // col[v]==1 找到奇环了
                int lc = LCA(u, v);
                // 把这个环的所有点染成一样的颜色, 当成一个点找增广路
                blossom(u, v, lc);  // 缩花
                blossom(v, u, lc);
            }
        }
    }
    return false;
}
signed main()
{
    // 随便匹配一些边
    for(auto [u, v]:e)if(!match[u] && !match[v]){
        match[u] = v, match[v] = u;
        ans++;
    }
    // 类似匈牙利, 每次从一个未匹配的点找增广路
    For(i,1,n)if(!match[i])
        ans += augment(i);
}
```

计算几何

从dls的板子改过来的(基本就改了下命名)

预备函数

```
#define db double
#define EPS 1e-9
#define PI acos(-1)
// 判断a的符号 返回值为 -1,0,1
inline int sign(db a) { return a < -EPS ? -1 : a > EPS; }
// 比较 a 和 b 的大小(带eps) a<b -1; a==b 0; a>b 1
inline int cmp(db a, db b) { return sign(a - b); }
// 点/向量
struct P{
    db x, y;
    int id;
    P(db _x = 0, db _y = 0) : x(_x), y(_y) {}

    P operator+(P p) const{ return P(x + p.x, y + p.y); }
    P operator-(P p) const{ return P(x - p.x, y - p.y); }
    P operator*(db a) { return P(x * a, y * a); }
    P operator/(db a) { return P(x / a, y / a); }
    // 按照pair(x,y)排序
    bool operator<(P p) const{
        int op = cmp(x, p.x);
        if (op != 0)return op == -1;
        return cmp(y, p.y) == -1;
    }
    // 两点重叠(带eps)
    bool operator==(P p) const { return cmp(x, p.x) == 0 && cmp(y, p.y) == 0; }
    bool operator!=(P p) const { return !((*this) == p);}
    // 点乘 (*this) · p
    db dotmul(P p) { return x * p.x + y * p.y; }
    // 叉乘   (*this) × p
    db xmul(P p) { return x * p.y - y * p.x; }
    // 模长平方
    db abs2() { return x * x + y * y; }
    double abs() { return sqrt(abs2()); }
    // 点到点p的距离
    double disTo(P p) { return (*this - p).abs(); }
    double disTo2(P p) { return (*this - p).abs2(); }
    // 极角 [-PI,PI]
    double alpha() { return atan2(y, x); }
```

```
    // 逆时针旋转90度
    P rota90() { return P(-y, x); }
    // 单位向量
    P unit() { return (*this) / abs(); }
    // 点是否在x轴上方 or 在x正半轴 (极角排序) 在上方返回1
    int quad() const { return sign(y) == 1 || (sign(y) == 0 && sign(x) >= 0); }
    // 逆时针旋转 a (弧度)
    P rota(db a) { return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)); }
    void read(){cin >> x >> y;}
    void print(){cout << "(" << x << ", " << y << ")\n";}
};
// 向量(p1p2) · 向量(p1p3)
db dotmul(P p1, P p2, P p3) {
    return (p2.x - p1.x) * (p3.x - p1.x) + (p2.y - p1.y) * (p3.y - p1.y);
}
// 向量(p1p2) × 向量(p1p3)
db xmul(P p1, P p2, P p3) {
    return (p2.x - p1.x) * (p3.y - p1.y) - (p3.x - p1.x) * (p2.y - p1.y);
}
// 叉乘的符号
int xmulOP(P p1, P p2, P p3) { return sign(xmul(p1, p2, p3)); }
// 向量夹角(p1->p2夹角, [-pi, pi])
db angle(P p1, P p2) {
  db a = p2.alpha() - p1.alpha();
  if(a < -PI) a += 2*PI;
  if(a > PI) a -= 2*PI;
  if(sign(a) < 0)a += 2*PI; // 加上这句话[0, 2*pi)
  return a;
}
```

直线 线段

```
// 直线p1p2 与 直线q1q2 是否相交
bool LLIsCross(P p1, P p2, P q1, P q2){
    db a1 = xmul(q1, q2, p1), a2 = -xmul(q1, q2, p2);
    return sign(a1 + a2);
}
// 直线p1p2 与 直线q1q2 的交点
P LLCross(P p1, P p2, P q1, P q2){
    db a1 = xmul(q1, q2, p1), a2 = -xmul(q1, q2, p2);
    return (p1 * a2 + p2 * a1) / (a1 + a2);
}

// 区间[l1,r1], [l2,r2]是否相交 (相交返回1)
bool intersect(db l1, db r1, db l2, db r2){
    if (l1 > r1) swap(l1, r1);
```

```
    if (l2 > r2) swap(l2, r2);
    return !(cmp(r1, l2) == -1 || cmp(r2, l1) == -1);
}
// 线段 p1p2, q1q2 是否相交
bool isSegCross(P p1, P p2, P q1, P q2){
    if (intersect(p1.x, p2.x, q1.x, q2.x) && intersect(p1.y, p2.y, q1.y, q2.y)
        && xmulOP(p1, p2, q1) * xmulOP(p1, p2, q2) <= 0
        && xmulOP(q1, q2, p1) * xmulOP(q1, q2, p2) <= 0)return true;
    return false;
}
// 线段 p1p2, q1q2 是否严格相交 交点不在端点
bool isSegCross_strict(P p1, P p2, P q1, P q2){
    if (xmulOP(p1, p2, q1) * xmulOP(p1, p2, q2) < 0
        && xmulOP(q1, q2, p1) * xmulOP(q1, q2, p2) < 0)return true;
    return false;
}

// m 是否在[a,b]之间
bool isInside(db a, db b, db m){
    if (a > b) swap(a, b);
    return cmp(a, m) <= 0 && cmp(m, b) <= 0;
}
// m 是否在a b为对角线的矩形区域内
bool isInside(P a, P b, P m){
    return isInside(a.x, b.x, m.x) && isInside(a.y, b.y, m.y);
}
// m 是否在线段ab上
bool isonSeg(P a, P b, P m){
    return xmulOP(a, b, m) == 0 && isInside(a, b, m);
}
// m是否严格在线段ab上
bool isonSeg_strict(P a, P b, P m){
    return xmulOP(a,b,m)==0
        && sign((m-a).dotmul(a-b))*sign((m-b).dotmul(a-b))<0;
}

// 点 q 到直线 p1p2 的投影 p1 != p2
P proj(P p1, P p2, P q){
    P dir = p2 - p1;
    return p1 + dir * (dir.dotmul(q - p1) / dir.abs2());
}
// 点 q 关于直线 p1p2 的轴对称点
P reflect(P p1, P p2, P q){
    return proj(p1, p2, q) * 2 - q;
}
// q 到线段 p1p2 的最小距离
db nearest(P p1, P p2, P q){
    if (p1 == p2) return p1.disTo(q);
```

```
    P h = proj(p1, p2, q);
    if (isInside(p1, p2, h)) return q.disTo(h);
    return min(p1.disTo(q), p2.disTo(q));
}
// 线段 p1p2 q1q2 的最小距离
db Segdis(P p1, P p2, P q1, P q2){
    if (isSegCross(p1, p2, q1, q2))return 0;
    return min(min(nearest(p1, p2, q1), nearest(p1, p2, q2)),
               min(nearest(q1, q2, p1), nearest(q1, q2, p2)));
}
//极角排序
// 要加const的话xmul也要加
bool cmp1(P a, P b){
    // 不在x轴同边 x轴上方和x正半轴排前
    if (a.quad() != b.quad())return a.quad() > b.quad();
    int dir = sign(a.xmul(b));      // b要在a的逆时针方向
    if(dir != 0) return dir > 0;
    return cmp(a.abs2(), b.abs2()) < 0;  //给个大小关系
};
```

多边形

面积、点包含

```
/**多边形端点都需要先极角排序**/
// 求多边形面积(简单多边形)
db area(vector<P> &ps){
    db res = 0;
    for (int i = 0; i < ps.size(); i++)
        res += ps[i].xmul(ps[(i + 1) % ps.size()]);
    return res / 2;
}
// 判断点p是否在多边形内 (2:内 1:在边上 0:外)(简单多边形)
int contain(const vector<P> &ps, P p){
    int n = ps.size(), res = 0;
    for (int i = 0; i < n; i++){
        P u = ps[i], v = ps[(i + 1) % n];
        if (isonSeg(u, v, p))return 1;
        if (cmp(u.y, v.y) <= 0)swap(u, v);
        if (cmp(p.y, u.y) > 0 || cmp(p.y, v.y) <= 0)continue;
        res ^= (xmulOP(p, u, v) > 0); //在p左边的有多少 p向左射线与多边形多少条边相交
    }
    return res * 2;
}
```

凸包

```
// 给定点集 求最小的多边形使得所有点在多边形上/内
vector<P> convexHull(vector<P> ps) { // 只要顶点
    int n = ps.size();
    if (n <= 1)return ps;
    sort(ps.begin(), ps.end()); // 按(x,y)字典序排 先排x再排y Point的小于号
    vector<P> res(n * 2);
    int k = 0;
    for (int i = 0; i < n; res[k++] = ps[i++]) //求下凸包
        while (k > 1 && xmulOP(res[k - 2], res[k - 1], ps[i]) <= 0)k--;
    for (int i = n - 2, t = k; i >= 0; res[k++] = ps[i--]) // 求上凸包
        while (k > t && xmulOP(res[k - 2], res[k - 1], ps[i]) <= 0)k--;
    res.resize(k - 1); // ps[0]被加入了2次(去掉一个)
    return res;
}
// 要顶点和边上的点
vector<P> convexHull_notStrict(vector<P> ps){
    // ps要去重
    int n = ps.size();
    if (n <= 1)return ps;
    sort(ps.begin(), ps.end()); // 按(x,y)字典序排 先排x再排y Point的小于号
    vector<P> res(n * 2);
    int k = 0;
    for (int i = 0; i < n; res[k++] = ps[i++]) //求下凸包
        while (k > 1 && xmulOP(res[k - 2], res[k - 1], ps[i]) < 0)k--;
    for (int i = n - 2, t = k; i >= 0; res[k++] = ps[i--]) // 求上凸包
        while (k > t && xmulOP(res[k - 2], res[k - 1], ps[i]) < 0)k--;
    res.resize(k - 1); // ps[0]被加入了2次(去掉一个)
    return res;
}
```

切凸多边形

```
// (逆时针) 用有向直线q1q2切凸多边形 返回切完后q1q2左边(逆时针方向)的多边形
vector<P> convexCut(const vector<P> &ps, P q1, P q2){
    vector<P> res;
    int n = ps.size();
    for (int i = 0; i < n; i++){
        P p1 = ps[i], p2 = ps[(i + 1) % n];
        int d1 = xmulOP(q1, q2, p1), d2 = xmulOP(q1, q2, p2);
        if (d1 >= 0)res.push_back(p1); // p1在直线左边
        if (d1 * d2 < 0)// p1p2 穿过直线 交点为新多边形的一个顶点
            res.push_back(LLCross(p1, p2, q1, q2));
    }
    return res;
}
```

旋转卡壳

```
// （逆时针） 凸多边形最远两点距离 / 凸多边形卡在两平行直线间 转一圈后 平行直线最大的距离
db convexDiameter(vector<P> ps){
    int n = ps.size();
    if (n <= 1)return 0;
    int is = 0, js = 0;
    for (int k = 1; k < n; k++){
        is = (ps[k] < ps[is] ? k : is); //选最大的
        js = (ps[js] < ps[k] ? k : js); //选最小的
    }
    int i = is, j = js;
    db res = ps[i].disTo(ps[j]);
    do{
        if ((ps[(i + 1) % n] - ps[i]).xmul(ps[(j + 1) % n] - ps[j]) >= 0)
            j = (j + 1) % n;
        else i = (i + 1) % n;
        res = max(res, ps[i].disTo(ps[j]));
    } while (i != is || j != js);
    return res;
}
```

最近点对

```
// 最近点对 ps先按x后按y排序
// min_dist(ps, 0, ps.size()-1)
db min_dist(vector<P>& ps, int l, int r)
{
    if(r-l<=5){
        db res = 1e100;
        For(i,l,r)For(j,i+1,r)
            res = min(res, ps[i].disTo(ps[j]));
        return res;
    }
    int m = (l+r)>>1;
    db res = min(min_dist(ps, l, m), min_dist(ps, m+1, r));
    vector<P> qs;
    For(i,l,r) if(abs(ps[i].x-ps[m].x) <= res)
        qs.push_back(ps[i]);
    sort(qs.begin(), qs.end(), [](P a, P b){return a.y<b.y;});
    For(i,1,(ll)qs.size()-1){
        for(int j = i-1; j>=0 && qs[j].y>=qs[i].y-res;j--)
            res = min(res, qs[i].disTo(qs[j]));
    }
    return res;
}
```

半平面交

```
struct Line{
    P a,b;
    Line(P _a=P(0,0), P _b=P(0,0)):a(_a),b(_b){}
    // 倾斜角(有向线段)
    double angle(){
        P v=b-a;
        return atan2(v.y,v.x);
    }
    //直线斜率排序(有向)
    bool operator < (const Line L)const{
        P va=b-a,vb=L.b-L.a;
        double A=atan2(va.y,va.x), B=atan2(vb.y,vb.x);
        if(cmp(A, B)==0)return va.xmul(L.b-a)>=0;
        return cmp(A,B)<0;
    }
};
// L1,L2 交点是否在 L0 右边
bool onRight(Line L0,P p){
    return xmulOP(L0.a,L0.b,p)<0;//三点呈顺时针
}
// 传入多边形的边 返回面积 面积为0点集不为空
double HalfPlaneInter(vector<Line> ls){
    sort(ls.begin(),ls.end());// 按斜率排序 按顺序切割
    int cnt=0,n=ls.size();
    for(int i = 0; i < n-1; i++){// 斜率相同的直线要靠左的一条
        if(cmp(ls[i].angle(), ls[i+1].angle())==0)continue;
        ls[cnt++]=ls[i];
    }
    ls[cnt++]=ls[n-1];
    ls.resize(cnt);
    deque<Line> q(n);
    deque<P> p(n);
    int front=0,last=0;//[front,last]
    q[0]=ls[0];//第一个半平面
    for(int i = 1; i < cnt; i++){
        while(last-front>0 && onRight(ls[i], p[last-1]))last--;
        while(last-front>0 && onRight(ls[i], p[front]))front++;
        q[++last]=ls[i];
        if(last-front>0)
            p[last-1]=LLCross(q[last-1].a, q[last-1].b, q[last].a, q[last].b);
    }
    while(q.size()>1 && onRight(q[front], p[last-1]))last--;
    if(last-front<=1)return -1;// 核不存在
    p[last]=LLCross(q[last].a, q[last].b, q[front].a, q[front].b);
    vector<P> ps;// 核
    for(int i = front; i <= last; i++)ps.push_back(p[i]);
```

```
    return area(ps); // 面积为0多边形可能存在一条线或一个点的核
}
```

三角形

```
// 三角形ABC 内心 (角平分线交点 内切圆)
P inCenter(P A, P B, P C){
    double a = (B - C).abs(), b = (C - A).abs(), c = (A - B).abs();
    return (A * a + B * b + C * c) / (a + b + c);
}
// 三角形ABC 外心 (中垂线交点 外接圆)
P outCenter(P A, P B, P C){
    P b = B - A, c = C - A;
    double d1 = b.abs2(), d2 = c.abs2(), d3 = 2 * b.xmul(c);
    return A - P(b.y * d2 - c.y * d1, c.x * d1 - b.x * d2) / d3;
}
// 三角形ABC 垂心 (垂线交点)
P orthoCenter(P A, P B, P C){
    P ba = B - A, ca = C - A, bc = B - C;
    double Y = ba.y * ca.y * bc.y,
           a = ca.x * ba.y - ba.x * ca.y,
           x0 = (Y + ca.x * ba.y * B.x - ba.x * ca.y * C.x) / a,
           y0 = -ba.x * (x0 - C.x) / ba.y + ca.y;
    return {x0, y0};
}
```

圆

```
// 圆o1 o2的关系 {0,1,2,3,4}分别为 {包含,内切,相交,外切,外离}
int type(P o1, db r1, P o2, db r2){
    db d = o1.disTo(o2);
    if (cmp(d, r1 + r2) == 1)return 4; // 外离
    if (cmp(d, r1 + r2) == 0)return 3; // 外切
    if (cmp(d, abs(r1 - r2)) == 1)return 2; // 相交
    if (cmp(d, abs(r1 - r2)) == 0)return 1; // 内切
    return 0;      // 包含
}
//圆与直线p1p2的交点 相切也返回两个点
vector<P> CirLine(P o, db r, P p1, P p2){
    if (cmp(abs((o - p1).xmul(p2 - p1) / p1.disTo(p2)), r) > 0)return {};
    db x=(p1-o).dotmul(p2-p1), y=(p2-p1).abs2(), d=x*x-y*((p1-o).abs2()-r*r);
    d = max(d, (db)0.0);
    P m = p1 - (p2-p1) * (x/y), dr = (p2-p1) * (sqrt(d)/y);
    return {m - dr, m + dr}; //沿着p1p2
}
// 两圆交点 要先判断两圆是否一样
```

```cpp
vector<P> CirCir(P o1, db r1, P o2, db r2){
    db d = o1.disTo(o2);
    if (cmp(d, r1 + r2) == 1)return {};
    if (cmp(d, abs(r1 - r2)) == -1)return {};
    d = min(d, r1 + r2);
    db y = (r1*r1 + d*d - r2*r2) / (2*d), x = sqrt(r1*r1 - y*y);
    P dr = (o2-o1).unit();
    P q1 = o1 + dr*y, q2 = dr.rota90() * x;
    return {q1 - q2, q1 + q2}; //沿着o1逆时针
}
// 求切线 r2:两圆外切线  -r2:两圆内切线   r2=0:点到圆的切线
vector<pair<P, P>> tanCC(P o1, db r1, P o2, db r2){
    P d = o2 - o1;
    db dr = r1 - r2, d2 = d.abs2(), h2 = d2 - dr * dr;
    if (sign(d2) == 0 || sign(h2) < 0)return {}; // o1==o2 || 内含 无切线
    h2 = max(0.0, h2);
    vector<pair<P, P>> res;
    for (db sign : {-1, 1}){
        // v:圆心垂直切线的单位方向向量
        P v = (d * dr + d.rota90() * sqrt(h2) * sign) / d2;
        res.push_back({o1 + v * r1, o2 + v * r2});
    }
    if (sign(h2) == 0)res.pop_back();
    return res;
}
```

最小圆覆盖

```cpp
// 找一个最小的圆可以覆盖所有点 O(n)
pair<P, db> min_circle(vector<P> ps){
    // 随机化 srand()一下
    shuffle(ps.begin(), ps.end(),
        mt19937{chrono::steady_clock::now().time_since_epoch().count()});
    int n = ps.size();
    P o = ps[0];
    db r = 0;
    for (int i = 1; i < n; i++)if (o.disTo(ps[i]) > r + EPS){
        o = ps[i], r = 0;
        for (int j = 0; j < i; j++)if (o.disTo(ps[j]) > r + EPS){
            o = (ps[i] + ps[j]) / 2;
            r = o.disTo(ps[i]);
            for (int k = 0; k < j; k++)if (o.disTo(ps[k]) > r + EPS){
                o = outCenter(ps[i], ps[j], ps[k]); //三角形外心
                r = o.disTo(ps[i]);
            }
        }
    }
    return {o, r};
}
```

圆与三角形交的面积

```
// 向量p1 p2的夹角
db rad(P p1, P p2) { return atan2l(p1.xmul(p2), p1.dotmul(p2)); }
// 圆心和三角形第三个顶点为原点  (unsure)
// 面积为有向面积 p1p2逆时针为正
db area_CirTri(db r, P p1, P p2){
    vector<P> is = CirLine(P(0, 0), r, p1, p2);
    if (is.empty())return r * r * rad(p1, p2) / 2;
    bool b1 = cmp(p1.abs2(), r * r) == 1, b2 = cmp(p2.abs2(), r * r) == 1;
    if (b1 && b2){ // 两点都在圆外
        if (sign((p1 - is[0]).dotmul(p2 - is[0])) <= 0 &&
            sign((p1 - is[0]).dotmul(p2 - is[0])) <= 0) // 在圆的不同边
            return r*r*(rad(p1, is[0]) + rad(is[1], p2))/2 +
is[0].xmul(is[1])/2;
        else return r * r * rad(p1, p2) / 2; // 在圆的同一边
    }
    // p1在圆外 is[0]是靠近p1的交点
    if (b1) return (r * r * rad(p1, is[0]) + is[0].xmul(p2)) / 2;
    // p2在圆外 is[0]是靠近p1的交点
    if (b2) return (r * r * rad(is[1], p2) + p1.xmul(is[1])) / 2;
    return p1.xmul(p2) / 2;  // 两点都在圆内 三角形面积
}
```

圆面积并

```
db intergal(db x, db y, db r, db L, db R){
    return r*r*(R-L) + x*r*(sinl(R)-sinl(L)) + y*r*(-cosl(R)+cosl(L));
}
db calc_area_cir(P c, db r, db L, db R){
    return intergal(c.x, c.y, r, L, R) / 2;
}
db norm(db x){ //将角度化到[0,2*PI]
    while (x < 0) x += 2 * PI;
    while (x > 2 * PI) x -= 2 * PI;
    return x;
}
P cs[N]; //圆心
db rs[N];     //圆的半径
int m;        //圆的数量
void work(){
    vector<int> cand;
    //去除包含的圆 重叠的保留1个
    for (int i = 0; i < m; i++){
        bool ok = 1;
        for (int j = 0; j < m; j++)if (i != j){
            //圆包含
```

```
            if (rs[j] > rs[i]+EPS && rs[i]+cs[i].disTo(cs[j]) <= rs[j]+EPS) {
                ok = 0;break;
            }
            if (cs[i] == cs[j] && cmp(rs[i], rs[j]) == 0 && j < i){ //圆重叠
                ok = 0;break;
            }
        }
        if (ok)cand.push_back(i);
    }
    m = cand.size();
    for (int i = 0; i < m; i++)cs[i] = cs[cand[i]], rs[i] = rs[cand[i]];

    db area = 0;

    for (int i = 0; i < m; i++){
        vector<pair<db, int>> ev = {{0, 0}, {2 * PI, 0}}; //扫描线
        int cur = 0;
        for (int j = 0; j < m; j++)if (j != i){
            vector<P> ret = CirCir(cs[i], rs[i], cs[j], rs[j]);
            if (!ret.empty()){
                db l = (ret[0] - cs[i]).alpha();
                db r = (ret[1] - cs[i]).alpha();
                l = norm(l), r = norm(r); //转化到[0,2*PI]
                ev.push_back({l, 1});
                ev.push_back({r, -1});
                if (l > r)++cur;
            }
        }

        sort(ev.begin(), ev.end());
        for (int j = 0; j < ev.size() - 1; j++){
            cur += ev[j].se;
            if (cur == 0){
                area += calc_area_cir(cs[i], rs[i], ev[j].fr, ev[j + 1].fr);
            }
        }
    }
    // return area;
}
```

最近点对

```
bool cmp_y(const P& a, const P& b) { return a.y < b.y; }
P a[N];
double mindist=1e99;
int ansa, ansb;// 最近点对的下标
```

```
void upd_ans(P pa,P pb){
    double dis = pa.disTo(pb);
    if(cmp(dis, mindist)<0)mindist=dis, ansa=pa.id, ansb=pb.id;
}
void NearestPoint(int l,int r){
    if(r-l<=3){
        for(int i = l; i <= r; i++)
            for(int j = i+1; j <= r; j++)
                upd_ans(a[i], a[j]);
        sort(a+l,a+r+1,cmp_y);return;
    }
    int mid = (l+r)>>1;
    db midx = a[mid].x;
    NearestPoint(l,mid);
    NearestPoint(mid+1,r);
    inplace_merge(a+l, a+mid+1, a+r+1, cmp_y);//归并两个数组

    static P t[N];
    int tsz=0;
    for (int i = l; i <= r; ++i){
        if (abs(a[i].x - midx) < mindist) {
            for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y < mindist; --j)
                upd_ans(a[i], t[j]);
            t[tsz++] = a[i];
        }
    }
}
void work(int n){
    sort(a+1,a+1+n);mindist=1e99;
    NearestPoint(1,n);
}
```

最小矩形覆盖

```
// 返回{面积,矩形(点逆时针 且y最小的在第一个)} (凸包ps)
pair<db,vector<P>> min_rectangle(vector<P> ps)
{
    vector<P> pans;
    int n=ps.size();
    for(int i = 0; i < n; i++)
        ps.push_back(ps[i]);//复制一份
    int l=0,u=0,r=0;
    P d=ps[1]-ps[0];// 最小矩形必有一条边与多边形一边重合
    for(int i = 0; i < n; i++){//找一个最初解
        if(cmp(d.dotmul(ps[i]), d.dotmul(ps[l]))<0)l=i;
        if(cmp(d.dotmul(ps[i]), d.dotmul(ps[r]))>0)r=i;
        if(cmp(d.xmul(ps[i]), d.xmul(ps[u]))>0)u=i;
    }
```

```
    db ans=1e99;//矩形面积
    for(int i = 0; i < n; i++){
        d=ps[i+1]-ps[i];
        while(cmp(d.dotmul(ps[l+1]), d.dotmul(ps[l]))<0)l++;//叉积最小
        while(cmp(d.dotmul(ps[r+1]), d.dotmul(ps[r]))>0)r++;//叉积最大
        while(cmp(d.xmul(ps[u+1]), d.xmul(ps[u]))>0)u++;// 最高的点
        assert(l<2*n && r<2*n && u<2*n);
        db x1=d.dotmul(ps[r]-ps[l])/d.abs();
        db x2=d.xmul(ps[u]-ps[i])/d.abs();
        db tmp=x1*x2;
        if(cmp(tmp,ans)<0){// 设ps[i]ps[i+1]矩形的右下边
            ans=tmp;
            P p1=LLCross(ps[i], ps[i]+d, ps[r], ps[r]+d.rota90());// 右顶点
            P p2=LLCross(ps[r], ps[r]+d.rota90(), ps[u], ps[u]-d);// 上顶点
            P p3=LLCross(ps[u], ps[u]-d, ps[l], ps[l]-d.rota90());// 左顶点
            P p4=LLCross(ps[l], ps[l]-d.rota90(), ps[i], ps[i]+d);// 下顶点
            pans = {p1,p2,p3,p4};// 已按逆时针
        }
    }
    auto cmp1 = [](const P a,const P b){
        if(cmp(a.y,b.y)==0)return cmp(a.x,b.x)<0;
        return cmp(a.y,b.y)<0;
    };
    //按y关键字排序最小的放第一个
    rotate(pans.begin(), min_element(pans.begin(),pans.end(),cmp1),
pans.end());
    return {ans,pans};
}
```

dp

决策单调性

对于转移代价$w(j,i)$表示j转移到i的代价，如果w满足四边形不等式，对于$a \le b \le c \le d$有

$$w(a,d)+w(b,c) \ge w(a,c)+w(b,d)$$

则dp满足决策单调性，对于$i<j$有$opt[i] \le opt[j]$，$opt[i]$为$dp[i]$的决策点

- 转移没有依赖的情况$f[i]=\min\limits_{j}(g[j]+w(i,j))$，其中$w(i,j)$为$j$转移到$i$的代价(为了看上去顺眼$w$的$ij$调转了)
  - 满足决策单调。
  - 复杂度为$O(n\log n)$

```
void solve(int l, int r, int optl, int optr){
    if(l>r)return;
    int mid = (l+r)>>1; // 算mid的opt
    int optm = optl;    // 随便定个决策点
    int rr = min(optr, mid-1);  // 根据题意来，这里表示mid的决策点一定在mid前面
    f[mid] = g[mid];        // 这应该是f[mid] = g[mid]+w(optl+1,mid)
    For(i,optl,rr){ // for在最优决策的区间
        if(f[mid] > g[i] + w(i+1, mid)){    // 如果用i更新mid更优，决策点更新为i
            f[mid] = g[i]+w(i+1, mid);
            optm = i;
        }
    }
    // 递归解决左右的f
    // 因为决策单调，左区间的决策点在[optl, optm]，右区间同理
    solve(l,mid-1,optl,optm);
    solve(mid+1,r,optm,optr);
}
```

- 转移有依赖的情况$f[i]=\min\limits_{0\le j<i}(f[j]+calc(i,j))$，其中$calc(i,j)$为$j$转移到$i$的代价
  - 同上面满足决策单调
  - 思路：使用一个队列维护$[i..n]$的决策点($[1..i-1]$已经计算了)。
  - 队列: $\{x_1,y_1\}\{x_2,y_2\}$，表示当前$i\in[x_1,x_2)$的决策点在y_1，$i\in[x_2,n]$的决策点在y_2。
  - 当新$f[i]$计算出来后，从后往前pop，
    - 如果$calc(x_2,i)<calc(x_2,y_2)$，表示$i$更新$[x_2,n]$更优，pop掉$\{x_2,y_2\}$
    - 如果更新x_2不优，需要在$[x_2,n]$二分出res，满足$[x_2,res)$用y_2更新更优，$[res,n]$用i更新更优，$\{res,i\}$入队

- 复杂度$O(n\log n)$

```cpp
int dp[N];
pii que[N];      // 维护i..n从哪转移最优
int head, tail; // 双端队列头尾
int que_size(){return tail-head+1;}

int calc(int i, int j)  // 从dp[j]转移到dp[i]的代价
{
    // int val = 1;
    // For(_,1,p){  // 这循环变量别写成ij
    //  val *= abs(pre[i]-pre[j]-l);
    // }
    // return dp[j] + val;
}

tail = -1, head = 0;     // 闭区间
que[++tail] = {1, 0};    // 1... 从0转移过来
For(i,1,n){
    if(que_size() && que[head].fr < i)que[head].fr++;
    // 如果队头区间空了就pop
    if(que_size()>=2 && que[head].fr >= que[head+1].fr)head++;
    dp[i] = calc(i, que[head].se);
    // cout << i << ' ' << dp[i] << ' ' << que[head].se << '\n';
    // 如果用i更新最后一段第一个数更优，就pop调最后一段
    while(que_size() && calc(que[tail].fr, que[tail].se)>=calc(que[tail].fr, i))
        tail--;
    if(que_size()){
        // 在最后一段上二分，找出i更新更好的后缀
        int l = que[tail].fr, r = n, res = n+1;
        // 最后一段表示的区间是[l, r]
        while(l<=r){
            int mid = (l+r)>>1;
            // i更新mid更优
            if(calc(mid, i) < calc(mid, que[tail].se)){
                res = mid;
                r = mid-1;
            }
            else l = mid+1;
        }
        if(res <= n)que[++tail] = {res, i};
    }
    else que[++tail] = {i, i};
}
```

字符串

kmp

```
// 1-base
void get_next(const string& s)
{
    nxt[1] = 0;
    nxt[2] = 1;
    int j = 1, i = 2, n = s.size()-1;
    while(i<=n){
        if(j==0 || s[i]==s[j]){
            i++; j++;
            nxt[i] = j;
        }
        else j = nxt[j];
    }
}
void kmp()
{
    string s, t;
    int n = s.size()-1, m = t.size()-1;
    get_next(s);
    For(i,1,m){
        while(j && t[i]!=s[j])j = nxt[j];
        if(t[i] == s[j])j++;
        // 出现一次
        if(j == n+1)x++, j = nxt[j];
        if(j==0)j++;
    }
}
void get_nxt_border(const string& ss){
    int len = (ll)ss.size()-1;
    for (int i=2;i<=len;i++){
        nxt[i] = nxt[i-1];
        while (nxt[i]&&ss[i]!=ss[nxt[i]+1]) nxt[i] = nxt[nxt[i]];
        nxt[i]+=(ss[i]==ss[nxt[i]+1]);
    }
}
```

exkmp/Z函数

```
// 1-base
// p的z函数，ext是s每个后缀和p的LCP
```

```
int ext[N], z[N];
void getZ(const string& p){
    // 先把1 2算了
    int n = (ll)p.size()-1;
    z[1] = n;
    while(z[2]+2 <= n && p[z[2]+1] == p[z[2]+2])z[2]++;
    int p0 = 2; // 记当前能扩到右边界是哪个后缀
    for(int i = 3; i <= n; i++){
        // 匹配没有到p0的又边界外
        if(i + z[i-p0+1] < p0 + z[p0])z[i] = z[i-p0+1];
        else {
            int cur = max(p0 + z[p0] - i, 0ll);
            // 暴力扩右边界
            while(cur + i <= n && p[cur+1] == p[cur+i])cur++;
            p0 = i; // 新右边界是当前后缀
            z[i] = cur;
        }
    }
}
void exkmp(const string& p, const string& s){
    getZ(p);
    int n = p.size()-1;
    int m = s.size()-1;
    // 暴力做整个串
    while(ext[1]+1 <= min(n, m) && p[ext[1]+1] == s[ext[1]+1])ext[1]++;
    int p0 = 1;
    For(i,2,m){      // 和前面getZ差不多
        if(i + z[i-p0+1] < p0 + ext[p0]) ext[i] = z[i-p0+1];
        else {
            int cur = max(p0 + ext[p0] - i, 0ll);
            while(cur+1 <= n && cur+i <= m && p[cur+1] == s[cur+i])cur++;
            ext[i] = cur;
            p0 = i;
        }
    }
}
```

border树

```

/*
查询前缀x，前缀y的最长公共border
*/
int nxt[N];
void get_nxt_border(const string& ss){
    int len = (ll)ss.size()-1;
    for (int i=2;i<=len;i++){
        nxt[i] = nxt[i-1];
```

```
        while (nxt[i]&&ss[i]!=ss[nxt[i]+1]) nxt[i] = nxt[nxt[i]];
        nxt[i]+=(ss[i]==ss[nxt[i]+1]);
    }
}
int fa[N][21], dep[N];
int lca(int x, int y){
    if(dep[x]<dep[y])swap(x, y);
    Rep(i,20,0)if(dep[fa[x][i]] >= dep[y])x = fa[x][i];
    if(x==y)return fa[x][0];     // 题目特判，真前缀
    Rep(i,20,0) if(fa[x][i] != fa[y][i])
        x = fa[x][i], y = fa[y][i];
    return fa[x][0];
}
void solve()
{
    int q, n;
    string s;
    cin >> s;
    n = s.size();
    s = " " + s;
    get_nxt_border(s.c_str());
    For(i,1,n){
        dep[i] = dep[nxt[i]]+1;
        fa[i][0] = nxt[i];
        For(j,1,20)fa[i][j] = fa[fa[i][j-1]][j-1];
    }
    cin >> q;
    while(q--){
        int x, y;
        cin >> x >> y;
        cout << lca(x, y) << "\n";
    }
}
```

马拉车 Manacher

```
int p[N];// 回文半径 [i-p[i]+1,i+p[i]-1]是回文串
int manacher(string& t)
{
    string s=" #";//每个字符间插个原串不存在的字符 头尾插一个
    for(int i = 0; i < t.size(); i++){
        s += t[i];
        s += "#";
    }
    int n=s.size();
    n--;
```

```
    For(i,1,n)p[i]=0;
    int m=0,r=0;
    for(int i = 1; i <= n; i++){
        if(i>r)p[i]=1;
        else p[i] = min(p[2*m-i], r-i+1); // i+p[i]-1 <= r
        while(i-p[i]>0 && i+p[i]<=n && s[i-p[i]]==s[i+p[i]]){//暴力扩
            p[i]++;
        }
        if(i+p[i]-1>r){//更新右端点和中点
            m=i;r=i+p[i]-1;
        }
    }
    int res = 0;//最大回文半径 对应插入后回文串长度为2*res-1  对应原串 res-1
    For(i,1,n){
        res = max(res,p[i]);
    }
    return res-1;
}
```

最小表示法(循环同构)

```
string getmin(string a)
{
    int n = a.size();
    a = " " + a + a;
    // cout << a << "\n";
    int i=1,j=2;
    while(j<=n){
        int k = 0;
        while(k<n && a[i+k]==a[j+k])k++;
        if(a[i+k]>a[j+k])i=i+k+1;
        else j=j+k+1;
        if(i==j)j++;
        if(i>j)swap(i,j);
    }
    string res = "";
    For(x,0,n-1){
        res += a[x+i];
    }
    return res;
}
```

后缀数组

```
/**
```

```
 * n 长度
 * m 字符集大小
 * s 字符串     0 base
 * sa 字典序第i小是哪个后缀
 * rk 后缀i的排名
 * ht lcp(sa[i], sa[i-1]);
*/
int sa[N], rk[N], ht[N];
void getSA(string& s, int m=128)
{
    static int xx[N], yy[N], c[N];
    int *x=xx, *y=yy;
    int n = s.size();
    for(int i = 0; i < m; i++) c[i] = 0;
    for(int i = 0; i < n; i++) c[x[i] = s[i]]++;
    for(int i = 1; i < m; i++) c[i] += c[i - 1];
    for(int i = n - 1; i >= 0; i--) sa[--c[x[i]]] = i;
    for(int k = 1; k < n; k <<= 1) {
        int p = 0;
        for(int i = n - 1; i >= n - k; i--) y[p++] = i;
        for(int i = 0; i < n; i++) if(sa[i] >= k) y[p++] = sa[i] - k;
        for(int i = 0; i < m; i++) c[i] = 0;
        for(int i = 0; i < n; i++) c[x[y[i]]]++;
        for(int i = 1; i < m; i++) c[i] += c[i - 1];
        for(int i = n - 1; i >= 0; i--) sa[--c[x[y[i]]]] = y[i];
        swap(x, y);
        p = 1; x[sa[0]] = 0; y[n] = -1;
        for(int i = 1; i < n; i++) {
            if(y[sa[i - 1]] == y[sa[i]] && y[sa[i - 1] + k] == y[sa[i] + k])
                x[sa[i]] = p - 1;
            else
                x[sa[i]] = p++;
        }
        if(p == n)break;
        m = p;
    }
    // 计算ht
    for(int i = 0; i < n; i++)rk[sa[i]] = i;
    int k = 0;
    for(int i = 0; i < n; i++) {
        k = max(k-1, 0ll);
        if(rk[i] == 0)continue;
        int j = sa[rk[i] - 1];
        while(s[i + k] == s[j + k]) k++;
        ht[rk[i]] = k;
    }
}
```

杂

高精度

```
namespace BigIntSP{
// 十进制高精度板子, 将 D 个十进制位压到一起
// FFT 时最好<=5
const int D = 1;
const int B = pow(10, D);
struct BigInt
{
    // ========================= 初始化部分 =========================
    // int, string 转 BigInt; BigInt 转 string 输出; 取绝对值, 取反; 高位推进

    int sign = 0;
    vector<ll> v;
    BigInt(ll x = 0){
        set(x);
    }
    BigInt(const string &s) {
        set(s);
    }
    void set(ll x){
        v.clear();
        sign = 0;
        if (x < 0) x *= -1, sign = 1;
        while (x) {
            v.push_back(x % B);
            x /= B;
        }
    }
    void set(const string& s){
        sign = 0;
        v.clear();
        int beg = 0;
        if (s[0] == '-')
            beg++, sign = 1;
        int add = 0, cnt = 0, base = 1;
        for (int i = s.size()-1; i >= beg; i--) {
            if (cnt == D) {
                v.push_back(add);
                cnt = add = 0;
                base = 1;
            }
            add = (s[i] - '0') * base + add;
```

```
            cnt++;
            base *= 10;
        }
        if (add) v.push_back(add);
    }
    BigInt operator-() const {
        BigInt res = *this;
        res.sign ^= 1;
        return res;
    }
    BigInt abs() const {
        BigInt res = *this;
        res.sign = 0;
        return res;
    }
    ll& operator[](const int i) { return v[i]; }
    int size() const { return v.size(); }
    void norm() {        // 向高位推一遍进位
        For(i,0,(ll)v.size()-2) {
            if (v[i] >= 0) {
                v[i + 1] += v[i] / B;
                v[i] %= B;
            } else {
                int c = (-v[i] + B - 1) / B;
                v[i] += c * B;
                v[i + 1] -= c;
            }
        }
        while (!v.empty() && v.back() >= B) {
            int c = v.back() / B;
            v.back() %= B;
            v.push_back(c);
        }
        while (!v.empty() && v.back() == 0) v.pop_back();
    }
    string to_str() const {
        string res;
        if (v.empty()) return "0";
        if (sign) res += '-';
        res += to_string(v.back());
        for (int i = (ll)v.size() - 2; i >= 0; i--) {
            string add;
            int w = v[i];
            For(_,1,D){
                add += ('0' + (w % 10));
                w /= 10;
            }
            reverse(all(add));
            res += add;
```

```
        }
        return res;
    }
    friend istream& operator>>(istream &is, BigInt &x) {
        string tmp;
        is >> tmp;
        x = BigInt(tmp);
        return is;
    }
    friend ostream& operator<<(ostream &os, BigInt x) {
        os << x.to_str();
        return os;
    }


    // ========================== O(n) 运算部分 ========================
    // 高精度加法, 高精度减法, 高精度乘低精度, 高精度除低精度

    BigInt& operator+=(const BigInt &x) {
        if (sign != x.sign) {
            *this -= (-x);
            return *this;
        }
        if ((int)v.size() < (int)x.size())
            v.resize(x.size(), 0);
        For(i,0,(ll)x.size()-1)
            v[i] += x.v[i];
        norm();
        return *this;
    }
    BigInt& operator-=(const BigInt &x) {
        if (sign != x.sign) {
            *this += (-x);
            return *this;
        }
        if (abs() < x.abs()) {
            *this = x - (*this);
            sign ^= 1;
            return *this;
        }
        For(i,0,(ll)x.size()-1)
            v[i] -= x.v[i];
        norm();
        return *this;
    }
    BigInt operator*(ll x) const {  // 注意爆ll (D<=9)
        BigInt res(*this);
        if (x < 0) res.sign ^= 1, x *= -1;
        for (int i = (ll)res.v.size()-1; i >= 0; i--)
```

```
            res.v[i] *= x;
        res.norm();
        return res;
    }
    BigInt& operator/=(ll x) {
        if (x < 0) sign ^= 1, x *= -1;
        for (int i = (ll)v.size()-1; i >= 0; i--){
            if (v[i] % x != 0 && i != 0) {
                v[i - 1] += B * (v[i] % x);
            }
            v[i] /= x;
        }
        norm();
        return *this;
    }

    BigInt operator+(const BigInt &x) const { return BigInt(*this) += x; }
    BigInt operator-(const BigInt &x) const { return BigInt(*this) -= x; }
    BigInt operator*=(const int &x) { return (*this) = (*this)*x; }
    BigInt operator/(const int &x) const { return BigInt(*this) /= x; }


    // ========================== 比较函数部分 ==========================

    bool gtZer(){return sign==0 && v.size();}   // return BitInt(x) > 0

    bool operator<(const BigInt &x) const {
        if (sign != x.sign) return sign > x.sign;
        if (v.size() != x.size()) {
            if (sign) return (int)x.size() < (int)v.size();
            else return (int)v.size() < (int)x.size();
        }
        Rep(i, (ll)v.size()-1, 0) if (v[i] != x.v[i]) {
            if (sign) return x.v[i] < v[i];
            else return v[i] < x.v[i];
        }
        return false;
    }
    bool operator>(const BigInt &x) const { return x < *this; }
    bool operator<=(const BigInt &x) const { return !(x < *this); }
    bool operator>=(const BigInt &x) const { return !(*this < x); }
    bool operator==(const BigInt &x) const { return !(*this < x) && !(x < *this); }
    bool operator!=(const BigInt &x) const { return !(*this == x); }


    // ========================== O(n^2) 运算部分 ==========================
    // 高精度乘高精度, 高精度除高精度, 高精度模高精度, 高精度取模, 高精度平方

```

```
    BigInt operator * (const BigInt &x) const {
        #ifndef UseFFT
        return n2Mul(*this, x);
        #else
        return FFTMul(*this, x);
        #endif
        // return karatsubaMul(x);
    }
    BigInt& operator/=(BigInt x){
        int lstSign = sign ^ x.sign;
        sign = x.sign = 0;
        if ((*this) < x)
            return *this = BigInt();
        if (x == BigInt(1)){
            sign = lstSign;
            return *this;
        }
        int d = v.size() - x.size() + 1;
        BigInt inv(1LL * B * B / x.v.back()), pre(0), c;
        int cur = 2, bcur = 1;
        while (inv != pre || bcur < x.size()) {
            bcur = min(bcur << 1, x.size());
            c.v = vector<ll>(x.v.end()-bcur, x.v.end());
            pre = inv;
            inv *= ((BigInt(2) << (cur + bcur - 1)) - inv * c);
            cur = min(cur << 1, d);
            inv.v = vector<ll>(inv.v.end()-cur, inv.v.end());
        }
        inv.v = vector<ll>(inv.v.end() - d, inv.v.end());
        BigInt res = (*this) * inv;
        res >>= (v.size());
        BigInt tt = (*this) - res * x;
        while (x <= tt) {
            res += BigInt(1);
            tt -= x;
        }
        v = res.v;
        return *this;
    }
    BigInt& operator%=(const BigInt &x) {
        BigInt div = (*this) / x;
        (*this) -= div * x;
        return *this;
    }

    void divMod(const BigInt& x, BigInt& divRes, BigInt& modRes) const {
        divRes = (*this) / x;
        modRes = (*this) - divRes*x;
    }
```

```

    BigInt square() {
        BigInt res = *this;
        res.sign = 0;
        #ifdef UseFFT
        auto v1 = FFT::vectorMul(v, v);
        #else
        auto v1 = n2Mul(*this, *this);
        #endif
        res.v.assign(v1.size(), 0);
        For(i, 0, (ll)v1.size()-1) {
            ll val = v1[i];
            for (int j = i; val; j++) {
                if (j == (int)res.v.size())
                    res.v.push_back(0);
                res.v[j] += val % B;
                val /= B;
            }
        }
        res.norm();
        return res;
    }

    BigInt operator*=(const BigInt &x) { return (*this) = (*this)*x; }
    BigInt operator/(const BigInt &x) const { return BigInt(*this) /= x; }
    BigInt operator%(const BigInt &x) const { return BigInt(*this) %= x; }

private:

    /****************三种乘法*********************/
    static BigInt n2Mul(const BigInt& a, const BigInt& b) {
        BigInt res;
        res.v.resize(a.size()+b.size(), 0);
        res.sign = a.sign ^ b.sign;
        for(int i = 0; i < a.size(); i++)
            for(int j = 0; j < b.size(); j++)
                res[i+j] += a.v[i]*b.v[j];
        res.norm();
        return res;
    }
    // BigInt& karatsubaMul(const BigInt& x){
    //     // todo
    // }
    #ifdef UseFFT
    static BigInt FFTMul(const BigInt& a, const BigInt& b) {
        if(a.size()+b.size() < 1000)return n2Mul(a, b);

        BigInt res;
        res.sign = a.sign ^ b.sign;
```

```
        auto v1 = FFT::vectorMul(a.v, b.v);
        res.v.assign(v1.size(), 0);
        for(int i = 0; i < v1.size(); i++){
            ll val = v1[i];
            for (int j = i; val; j++) {
                if (j == (int)res.v.size())
                    res.v.push_back(0);
                res.v[j] += val % B;
                val /= B;
            }
        }
        res.norm();
        return res;
    }
    #endif

    /*** 这的左右移不是正常的左右移 做除法时用 ***/
    BigInt& operator<<=(const int& x){
        if(!v.empty()) {
            vector<ll> add(x, 0);
            v.insert(v.begin(), all(add));
        }
        return *this;
    }
    BigInt& operator>>=(const int& x){
        v = vector<ll>(v.begin()+min(x,(int)v.size()), v.end());
        return *this;
    }
    BigInt operator<<(const int& x)const{return BigInt(*this)<<=x;}
    BigInt operator>>(const int& x)const{return BigInt(*this)>>=x;}
};
typedef BigInt Bigint;
Bigint qmi(Bigint m, int k)
{
    Bigint res(1);
    while (k) {
        if (k & 1) res *= m;
        m *= m;
        k >>= 1;
    }
    return res;
}
}
using BigIntSP::BigInt;
using BigIntSP::qmi;
```

整数三分

```
int l = -1e18, r = 1e18;
    while(r-l>5){
        int midl = (r+l)/2, midr = (midl+r)/2;
        db resl = check(midl), resr = check(midr);
        if(resl < resr){
            r = midr;
        }
        else l = midl;
    }
    db ans = 1e18;
    For(i,l,r){
        ans = min(ans, check(i));
    }
```

日期

```
struct Date{
    int year, month, day;
};
int days[12]={31,28,31,30,31,30,31,31,30,31,30,31};

// 闰年
inline int leap(int year){
    return (year%4==0&&year%100!=0)||year%400==0;
}

//返回指定日期是星期几
int weekday(Date a){
    int tm = a.month>=3 ? (a.month-2) : (a.month+10);
    int ty = a.month>=3 ? a.year : (a.year-1);
    return (ty+ty/4-ty/100+ty/400+(int)(2.6*tm-0.2)+a.day)%7;
}

// 日期转天数 0/1/1 为第一天
int date2int(Date a){
    int ret = a.year*365+(a.year-1)/4-(a.year-1)/100+(a.year-1)/400;
    days[1] += leap(a.year);
    for(int i=0; i<a.month-1; ret+=days[i++]){}
    days[1] = 28;
    return ret+a.day;
}

//天数偏移转日期
Date int2date(int a){
    Date ret;
    ret.year = a/146097*400;
```

```
    for(a%=146097; a>=365+leap(ret.year); a-=365+leap(ret.year),ret.year++);
    days[1] += leap(ret.year);
    for(ret.month=1; a>=days[ret.month-1]; a-=days[ret.month-1],ret.month++);
    days[1] = 28;
    ret.day = a+1;
    return ret;
}
```

模拟退火

可调:

初始温度 T

结束温度 eps

降温系数 一般为接近1的数字

参考:

10^{-1}精度: 下界10^{-3}或10^{-4} 降温系数0.8

10^{-5}精度: 下界10^{-7}或10^{-8} 降温系数0.99

```
struct Point {
    double x, y, z;
    Point(){}
    Point(double _x, double _y, double _z):x(_x),y(_y),z(_z){}
    bool operator == (Point b){
        return abs(x-b.x)<1e-6 && abs(y-b.y)<1e-6 && abs(z-b.z)<1e-6;
    }
};
double dis(Point a, Point b) {
    double dx = a.x-b.x, dy = a.y-b.y, dz = a.z-b.z;
    return sqrt(dx*dx+dy*dy+dz*dz);
}
Point p[N];
int n;
double maxdis(Point a)// 求函数值

bool out(Point a){}// 判断自变量越界

Point ansp;
double ans;
```

```
void SA(){  // 模拟退火
    double T = 550, eps = 1e-10;
    while(T>eps){   // 结束温度
        Point tmp(ansp);
        // 随机到的点在答案范围内 (也可以不加，温度高容易随机到边界外 可能会卡死)
        while(tmp == ansp || out(tmp)){
            tmp.x = ansp.x + T*(2*rand()-RAND_MAX);// 根据当前温度随机选
            tmp.y = ansp.y + T*(2*rand()-RAND_MAX);
            tmp.z = ansp.z + T*(2*rand()-RAND_MAX);
        }
        double tmpans = maxdis(tmp);    // 计算答案
        if(tmpans - ans < 1e-6){        // 比当前答案更优 更新答案
            ans = tmpans;
            ansp=tmp;
        }
        // 比当前答案不优 有一定概率选择不优的答案
        else if(exp(-abs(tmpans-ans)/T) * RAND_MAX > rand()){
            ans = tmpans;
            ansp=tmp;
        }
        T *= 0.998; //降温系数
    }
}
void tuihuo() {
    ansp = Point(0,0,0);    // 随便指定一个答案
    ans = maxdis(ansp);
    // 做多几次退火
    while(clock() < 1000)SA();      // 卡时间
    // for(int i = 0; i < 100; i++)SA();    // 固定次数
}
```

C++函数

- （C++17以后）第一类不完全椭圆积分：`double ellint_1(double k, double phi);`，k为椭圆离心率，phi为雅可比振幅(弧度)。返回值为$\int_0^{phi} \frac{1}{\sqrt{1-k^2\sin^2\theta}}\,d\theta$。
- （C++17以后）第二类不完全椭圆积分：`double ellint_2(double k, double phi);`，k, phi同上。返回值为$\int_0^{phi} \sqrt{1-k^2\sin^2\theta}\,d\theta$。
- （C++17以后）第三类不完全椭圆积分：`double ellint_3(double k, double nu, double phi);`，k, phi同上，nu椭圆特征(公式中的n)。返回值为$\int_0^{phi} \frac{1}{(1-n\sin^2\theta)\sqrt{1-k^2\sin^2\theta}}\,d\theta$。

python函数

`ord()`字符转他的ASCII码，`chr()`ASCII码转字符

字符串排序`s = ''.join(sorted(s))`

分数:

```
from fractions import Fraction
x = Fraction(1, 2) # 1/2
print(x.numerator, x.denominator)
# 分子, 分母
```

快读快写 & debug

```
void debug(){cerr << "\n";}
template<typename T, typename... Args>
void debug(T head, Args... tail){
    cerr << head << " ";
    debug(tail...);
}
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "]: ", debug(__VA_ARGS__)

template<typename T>
inline void read(T &x) {
    x=0;bool flag(0);char ch=getchar();
    while(!isdigit(ch)) flag=ch=='-',ch=getchar();
    while(isdigit(ch)) x=(x<<1)+(x<<3)+(ch^48),ch=getchar();
    flag?x=-x:0;
}
template<typename T>
inline void write(T x,bool mode=1)//0为空格，1为换行 {
    x<0?x=-x,putchar('-'):0;static short Stack[50],top(0);
    do Stack[++top]=x%10,x/=10; while(x);
    while(top) putchar(Stack[top--]|48);
    mode?putchar('\n'):putchar(' ');
}
```

卡常工具

pbds(更快的哈希...)

```
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>//用tree
#include<ext/pb_ds/hash_policy.hpp>//用hash
#include<ext/pb_ds/trie_policy.hpp>//用trie
#include<ext/pb_ds/priority_queue.hpp>//用priority_queue
using namespace __gnu_pbds;

```

```
cc_hash_table<int,bool> h;
gp_hash_table<int,bool> h;  // 优先用

#define pii pair<int,int>
#define mp(x,y) make_pair(x,y)
tree<pii,null_type,less<pii>,rb_tree_tag,tree_order_statistics_node_update> tr;
pii //存储的类型
null_type //无映射(低版本g++为null_mapped_type)
less<pii> //从小到大排序
rb_tree_tag //红黑树
tree_order_statistics_node_update //更新方式
tr.insert(make_pair(x,y)); //插入;
tr.erase(make_pair(x,y)); //删除;
tr.order_of_key(pii(x,y)); //求排名
tr.find_by_order(x); //找k小值，返回迭代器
tr.join(b); //将b并入tr，前提是两棵树类型一样且没有重复元素
tr.split(v,b); //分裂，key小于等于v的元素属于tr，其余的属于b
tr.lower_bound(x); //返回第一个大于等于x的元素的迭代器
tr.upper_bound(x); //返回第一个大于x的元素的迭代器
//以上所有操作的时间复杂度均为O(logn)
```

对拍(linux)

```
import os
code, std, gen, tc = 'B', 'std', 'zgen', 0
os.system(f'g++ {code}.cpp -o {code}')
os.system(f'g++ {std}.cpp -o {std}')
os.system(f'g++ {gen}.cpp -o {gen}')
print('compile OK')
while True:
    tc += 1
    os.system(f"./{gen}>in.txt")
    os.system(f"./{std}<in.txt>ans.txt")
    os.system(f"./{code}<in.txt>out.txt")
    # -b不检查空格不同 -B不检查空行不同，-q不显示详细信息
    if os.system("diff -b -B out.txt ans.txt"):
        print('WA')
        break
    print(f"ok in case {tc}")
```

vscode配置

Bracket Pair Colorization (是否对括号着色)

方向键: cursor(Up/Down...)

Editor:Insert space (用\t缩进)

扩展>Run Code configuration>

Run In Terminal（在终端运行）

Save (All) Files Before Run

Ignore Selection

```
    executorMap:`g++ $fileName -o $fileNameWithoutExt -std=c++17 -O2 -Wall -Wl,--stack=1073741824
                 python3 -u
```

可选编译参数

```
-D_GLIBCXX_DEBUG :STL debugmode
-fsanitize=address:内存错误检查
-fsanitize=undefined :UB检查

```

最后

- 在`long long`范围内，用`long double`代替`long long`不会丢精度
- `x>>i`，`i`不能超过`x`的类型的字长，否则可能会寄
- builtin的函数如果类型为ll要使用对应的函数`__builtin_popcountll()`
- 浮点数二/三分要写次数